

CUSTOMER PORTAL

S. No	PAGES
1	LOGIN
2	PROFILE
3	DASHBOARD
4	INQUIRY
5	SALES ORDERS
6	DELIVERY
7	INVOICE , INVOICE PDF
8	PAYMENT AND AGING
9	CREDIT AND DEBIT MEMO
10	OVERALL SALES

LOGIN

[illegible]

```
FUNCTION zfm_sd_rk_custlogininvalidate.  
*"-----  
*****Local Interface:  
*" IMPORTING  
*"      VALUE(IV_KUNNR) TYPE KUNNR  
*"      VALUE(IV_PASSWORD) TYPE Z_RK_PWDHASH_DE  
*" EXPORTING  
*"      VALUE(EV_LOGINSUCCESS) TYPE STRING  
*"      VALUE(EV_MESSAGE) TYPE STRING  
*"      VALUE(EV_CUSTOMER_NAME) TYPE STRING  
*"-----  
  
DATA: loc_login_attempts TYPE i,  
      loc_name1          TYPE name1,  
      wa_custcreds       TYPE zsd_rk_custcreds.  
  
SELECT SINGLE name1 FROM kna1  
    INTO @loc_name1  
    WHERE kunnr = @iv_kunnr.  
  
IF sy-subrc <> 0 OR loc_name1 IS INITIAL.  
    ev_loginsuccess = 'N'.  
    ev_message = 'Customer does not exist'.  
    RETURN.  
ENDIF.  
  
SELECT SINGLE * FROM zsd_rk_custcreds  
    INTO @wa_custcreds  
    WHERE kunnr = @iv_kunnr.  
  
IF sy-subrc <> 0.  
    ev_loginsuccess = 'N'.  
    ev_message = 'No login credentials found for customer'.  
    RETURN.  
ENDIF.  
  
IF wa_custcreds-login_attempts <= 0.  
    ev_loginsuccess = 'N'.  
    ev_message = 'Login attempts exceeded. Please contact your IT  
administrator.'.  
    RETURN.
```

```

ENDIF.

IF wa_custcreds-password = iv_password.
    ev_loginsuccess = 'Y'.
    ev_message = 'Login successful'.
    ev_customer_name = loc_name1.

    UPDATE zsd_rk_custcreds
        SET login_attempts = 5
        WHERE kunnr = @iv_kunnr.

ELSE.
    loc_login_attempts = wa_custcreds-login_attempts - 1.

    UPDATE zsd_rk_custcreds
        SET login_attempts = @loc_login_attempts
        WHERE kunnr = @iv_kunnr.

    IF loc_login_attempts <= 0.
        ev_loginsuccess = 'N'.
        ev_message = 'Login attempts exceeded. Please contact your IT
administrator.'.
    ELSE.
        ev_loginsuccess = 'N'.
        ev_message = |Incorrect password. Remaining attempts: {
loc_login_attempts } of 5.|.
    ENDIF.

ENDIF.

ENDIF.

ENDFUNCTION.

```

Access Url:

SAP_GET_AUTH_URL =
http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_sd_rk_customer_auth?sap-client=100

PROFILE

Structure Name: **ZSD_RK_CUSTOMER_PROFILE_S**

Structure Field	Data Element	Table	Field Name in Table	Description
WF_KUNNR	KUNNR	KNA1	KUNNR	Customer Number
WF_NAME1	NAME1_GP	KNA1	NAME1	Name 1
WF_SPRAS	SPRAS	KNA1	SPRAS	Language Key
WF_ERDAT	ERDAT	KNA1	ERDAT	Created On
WF_ERNAM	ERNAM	KNA1	ERNAM	Name of Person Who Created the Object
WF_STREET	STREET	ADRC	STREET	Street
WF_CITY1	CITY1	ADRC	CITY1	City
WF_POST_CODE1	PSTCD1	ADRC	POST_CODE1	Postal Code
WF_COUNTRY	LAND1	ADRC	COUNTRY	Country Key
WF_TIME_ZONE	TZONE	ADRC	TIME_ZONE	Time Zone
WF_VKORG	VKORG	KNVV	VKORG	Sales Organization
WF_VTWEG	VTWEG	KNVV	VTWEG	Distribution Channel
WF_WAERS	WAERS	KNVV	WAERS	Currency Key
WF_BZIRK	BZIRK	KNVV	BZIRK	Sales District

Function Module Code

```
FUNCTION zfm_sd_rk_get_custprofile.  
*-----  
*""Local Interface:  
*  IMPORTING  
*      VALUE(IV_KUNNR) TYPE  KUNNR  
*  EXPORTING  
*      VALUE(EV_CUSTOMER_PROFILE) TYPE  ZSD_RK_CUSTOMER_PROFILE_S  
*      VALUE(EV_MESSAGE) TYPE  STRING  
*-----  
  
  DATA: wa_knal TYPE knal,  
         wa_adrc TYPE adrc,  
         wa_knvv TYPE knvv.  
  
  " Step 1: Get general customer data  
  SELECT SINGLE * INTO wa_knal FROM knal WHERE kunnr = iv_kunnr.  
IF sy-subrc <> 0.  
  ev_message = |Customer with ID ({ iv_kunnr }) is not found|.br/>  RETURN.  
ENDIF.  
  
  " Step 2: Get address data via ADRNR
```

```

SELECT SINGLE street, city1, post_code1, country, time_zone
  INTO CORRESPONDING FIELDS OF @wa_adrc
FROM adrc
WHERE addrnumber = wa_knal-adrnr.

IF sy-subrc <> 0.
  CLEAR wa_adrc.
ENDIF.

" Step 3: Get sales area data
SELECT SINGLE * INTO wa_knvv FROM knvv WHERE kunnr = iv_kunnr.
IF sy-subrc <> 0.
  CLEAR wa_knvv.
ENDIF.

" Step 4: Fill output structure
CLEAR ev_customer_profile.

ev_customer_profile-wf_kunnr      = wa_knal-kunnr.
ev_customer_profile-wf_name1     = wa_knal-name1.
ev_customer_profile-wf_spras     = wa_knal-spras.
ev_customer_profile-wf_erdat     = wa_knal-erdat.
ev_customer_profile-wf_ernam     = wa_knal-ernam.

ev_customer_profile-wf_street    = wa_adrc-street.
ev_customer_profile-wf_city1     = wa_adrc-city1.
ev_customer_profile-wf_post_code1 = wa_adrc-post_code1.
ev_customer_profile-wf_country   = wa_adrc-country.
ev_customer_profile-wf_time_zone = wa_adrc-time_zone.

ev_customer_profile-wf_vkorg     = wa_knvv-vkorg.
ev_customer_profile-wf_vtweg    = wa_knvv-vtweg.
ev_customer_profile-wf_waers     = wa_knvv-waers.
ev_customer_profile-wf_bzirk     = wa_knvv-bzirk.

ENDFUNCTION.

```

Access URL:

**[http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_sd_rk_get_customerprofile?
sap-client=100](http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_sd_rk_get_customerprofile?sap-client=100)**

SALES & DISTRIBUTION

INQUIRY

Structure Name: **ZSD_RK_INQUIRY_HEADER_S**

Table Type Name: **ZSD_RK_INQUIRY_HEADERS_T**

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_VBELN	VBELN	VBAK	VBELN	Inquiry Number – Sales Document Number
WF_ERDAT	ERDAT	VBAK	ERDAT	Created On – Date on which the inquiry was created
WF_AUDAT	AUDAT	VBAK	AUDAT	Document Date in Document – Date on the inquiry itself
WF_ERNAM	ERNAM	VBAK	ERNAM	Created By – User who created the inquiry record
WF_KUNNR	KUNNR	VBAK	KUNNR	Customer Number – Sold-to party for this inquiry
WF_VKORG	VKORG	VBAK	VKORG	Sales Organization – Responsible sales unit
WF_VTWEG	VTWEG	VBAK	VTWEG	Distribution Channel – Channel through which sold
WF_SPART	SPART	VBAK	SPART	Division – Business division
WF_VKBUR	VKBUR	VBAK	VKBUR	Sales Office – Office responsible for this inquiry
WF_VKGRP	VKGRP	VBAK	VKGRP	Sales Group – Sales group within the sales office
WF_BSTNK	BSTNK	VBAK	BSTNK	Purchase Order Number – PO number from customer (if any)
WF_ANGDT	ANGDT	VBAK	ANGDT	Quotation Date – Date of inquiry creation (since AUART = AF)
WF_BNDDT	BNDDT	VBAK	BNDDT	Quotation Valid-To Date – Validity end date of the inquiry
WF_WAERK	WAERK	VBAK	WAERK	Currency Key – Currency in which inquiry net values are stored
WF_AUART	AUART	VBAK	AUART	Sales Document Type – ‘AF’ (Inquiry)
WF_AUGRU	AUGRU	VBAK	AUGRU	Order Probability – Probability key for inquiry (e.g., “10%”, “50%”)
WF_TOTAL_NETWR	NETWR	VBAK	NETWR	Net Order Value – Total net value of inquiry header (from VBAK)

WF_VKORG_TXT	VTEXT	TVKOT	VTEXT	Sales Org. Description – Language-specific text for VKORG
WF_VTWEG_TXT	VTEXT	TVTWT	VTEXT	Distr. Channel Description – Language-specific text for VTWEG
WF_SPART_TXT	VTEXT	TSPAT	VTEXT	Division Description – Language-specific text for SPART
WF_VKBUR_TXT	BEZEI	TVKBT	BEZEI	Sales Office Description – Text for VKBUR
WF_VKGRP_TXT	BEZEI	TVGRT	BEZEI	Sales Group Description – Text for VKGRP
WF_TOTAL_ITEMS	INT4 (“I”)	– (calculate d)	–	Total Number of Items – Count of line items in this inquiry
WF_TOTAL_NETWR	P (e.g. DEC15_2)	– (calculate d)	–	Total Net Value – Sum of net values for all inquiry items
WF_ORDERED_ITEMS	INT4 (“I”)	– (calculate d)	–	Items Linked to Orders – Count of how many items turned into a sales order
WF_STATUS_TEXT	CHAR10	– (calculate d)	–	Inquiry Status – Literal “OPEN” / “PARTIAL” / “FULL”
WG_FULLY_ORDERED	CHAR1	– (calculate d)	–	Fully Ordered Flag – ‘X’ if all items have been ordered
WG_PARTIALLY_ORDE RED	CHAR1	– (calculate d)	–	Partially Ordered Flag – ‘X’ if some (but not all) items are ordered

Structure Name: **ZSD_RK_INQUIRY_ITEM_S**

Table Type Name: **ZSD_RK_INQUIRY_ITEM_T**

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_VBELN	VBELN	VBAP	VBELN	Inquiry Number – Sales Document Number (header reference)
WF_POSNR	POSNR	VBAP	POSNR	Item Number – Position in the inquiry
WF_MATNR	MATNR	VBAP	MATNR	Material Number – Material being inquired about
WF_ARKTX	ARKTX	VBAP	ARKTX	Short Text – Item description (free-text)

WF_KWMENG	KWMENG	VBAP	KWMENG	Quantity in Order Unit – Quantity requested
WF_VRKME	VRKME	VBAP	VRKME	Sales Unit – Unit of measure
WF_NETPR	NETPR	VBAP	NETPR	Net Price – Unit price (per VRKME)
WF_NETWR	P (e.g. DEC15_2)	– (calculated)	–	Net Value – Computed: KWMENG × NETPR
WF_WAERK	WAERK	VBAP	WAERK	Currency Key – Currency in which NETPR is expressed
WG_IS_ORDERED	CHAR1	– (calculated)	–	Ordered Flag – 'X' if this inquiry item was converted to a sales order
WF_VBELN_SO	VBELN	VBFA (calculated)	VBELN (vbfa-vbeln)	Sales Order Number – If converted, the resulting SO number
WF_POSNR_SO	POSNR	VBFA (calculated)	POSNN	Sales Order Item No – If converted, the item position in SO



Function Module to Return Inquiry Data

Function Module Name: ZFM_SD_RK_GET_INQUIRY_DATA

FUNCTION zfm_sd_rk_get_cust_inquiry.

```

* "-----
* " * "Local Interface:
* "   IMPORTING
* "       VALUE(IV_KUNNR) TYPE KUNNR
* "       VALUE(IV_DATE_FROM) TYPE ERDAT DEFAULT 00000000
* "       VALUE(IV_DATE_TO) TYPE ERDAT DEFAULT 99991231
* "       VALUE(IV_SORT_NEWEST) TYPE STRING OPTIONAL
* "       VALUE(IV_PAGE_NUM) TYPE I DEFAULT 1
* "       VALUE(IV_PAGE_SIZE) TYPE I DEFAULT 10
* "       VALUE(IV_VBELN) TYPE VBELN_VA OPTIONAL
* "       VALUE(IV_STATUS) TYPE STRING OPTIONAL
* "   EXPORTING
* "       VALUE(ET_INQUIRY_HEADERS) TYPE ZSD_RK_INQUIRY_HEADER_T
* "       VALUE(ET_INQUIRY_ITEMS) TYPE ZSD_RK_INQUIRY_ITEM_T
* "       VALUE(EV_TOTAL_RECORDS) TYPE I
* "       VALUE(EV_MESSAGE) TYPE STRING
* "-----

" 1. Input Validation
IF iv_kunnr IS INITIAL.
    ev_message = 'Customer number is required'.
    RETURN.
ENDIF.

```



```

TYPES: BEGIN OF ty_vbfa_partial,
        vbelv TYPE vbfa-vbelv,
        posnv TYPE vbfa-posnv,
        vbeln TYPE vbfa-vbeln,
        posnn TYPE vbfa-posnn,
      END OF ty_vbfa_partial.

DATA: it_vbfa TYPE TABLE OF ty_vbfa_partial,
      wa_vbfa TYPE ty_vbfa_partial.

DATA: loc_vkorg_txt TYPE tvkot-vtext,
      loc_vtweg_txt TYPE tvtw-tvtext,
      loc_spart_txt TYPE tspat-vtext,
      loc_vkbur_txt TYPE tvkbt-bezei,
      loc_vkgrp_txt TYPE tvkgt-bezei.

DATA: loc_total_items TYPE i VALUE 0,
      loc_total_netwr TYPE netwr VALUE 0,
      loc_items_linked TYPE i VALUE 0,
      loc_item_value   TYPE p LENGTH 15 DECIMALS 2.

DATA: wg_is_ordered TYPE char1.

DATA: it_temp_items TYPE zsd_rk_inquiry_item_t.

DATA: loc_vbeln_so TYPE vbeln,
      loc_posnr_so TYPE posnr.

DATA: it_vbak TYPE STANDARD TABLE OF vbak WITH EMPTY KEY.

DATA(loc_date_from) = COND erdat( WHEN iv_date_from IS INITIAL THEN
'00000000' ELSE iv_date_from ).
DATA(loc_date_to)   = COND erdat( WHEN iv_date_to IS INITIAL THEN
'99991231' ELSE iv_date_to ).

DATA(loc_offset) = ( iv_page_num - 1 ) * iv_page_size.

IF iv_vbeln IS NOT INITIAL.
  SELECT vbeln, erdat, audat, ernam, kunnr, vkorg, vtweg, spart,
         vkbur, vkgrp, bstnk, angdt, bnndt, waerk, auart, augru, netwr
  FROM vbak
  INTO CORRESPONDING FIELDS OF TABLE @it_vbak
  WHERE vbeln = @iv_vbeln
         AND auart = 'AF'.
ELSEIF iv_status IS INITIAL.
  IF iv_sort_newest = 'X'.
    SELECT vbeln, erdat, audat, ernam, kunnr, vkorg, vtweg, spart,
           vkbur, vkgrp, bstnk, angdt, bnndt, waerk, auart, augru, netwr
    FROM vbak
    WHERE auart = 'AF'
           AND kunnr = @iv_kunnr
           AND audat BETWEEN @loc_date_from AND @loc_date_to
    ORDER BY audat DESCENDING
    INTO CORRESPONDING FIELDS OF TABLE @it_vbak
    UP TO @iv_page_size ROWS
    OFFSET @loc_offset.
  ELSE.

```

```

SELECT vbeln, erdat, audat, ernam, kunnr, vkorg, vtweg, spart,
       vkbur, vkgrp, bstnk, angdt, bnddt, waerk, auart, augru, netwr
FROM vbak
WHERE auart = 'AF'
      AND kunnr = @iv_kunnr
      AND audat BETWEEN @loc_date_from AND @loc_date_to
ORDER BY audat ASCENDING
INTO CORRESPONDING FIELDS OF TABLE @it_vbak
UP TO @iv_page_size ROWS
OFFSET @loc_offset.
ENDIF.
ELSE.
IF iv_sort_newest = 'X'.
SELECT vbeln, erdat, audat, ernam, kunnr, vkorg, vtweg, spart,
       vkbur, vkgrp, bstnk, angdt, bnddt, waerk, auart, augru, netwr
FROM vbak
INTO CORRESPONDING FIELDS OF TABLE @it_vbak
WHERE auart = 'AF'
      AND kunnr = @iv_kunnr
      AND audat BETWEEN @loc_date_from AND @loc_date_to
ORDER BY audat DESCENDING.
ELSE.
SELECT vbeln, erdat, audat, ernam, kunnr, vkorg, vtweg, spart,
       vkbur, vkgrp, bstnk, angdt, bnddt, waerk, auart, augru, netwr
FROM vbak
INTO CORRESPONDING FIELDS OF TABLE @it_vbak
WHERE auart = 'AF'
      AND kunnr = @iv_kunnr
      AND audat BETWEEN @loc_date_from AND @loc_date_to
ORDER BY audat ASCENDING.
ENDIF.
ENDIF.

" Exit if No Data Found
IF it_vbak IS INITIAL.
  ev_message = 'No inquiries found for specified criteria'.
  ev_total_records = 0.
  RETURN.
ENDIF.

IF iv_vbeln IS NOT INITIAL.
  ev_total_records = lines( it_vbak ).
ELSEIF iv_status IS INITIAL.
  SELECT COUNT(*) INTO @ev_total_records
  FROM vbak
  WHERE auart = 'AF'
        AND kunnr = @iv_kunnr
        AND audat BETWEEN @loc_date_from AND @loc_date_to.
ELSE.
  ev_total_records = lines( et_inquiry_headers ).
ENDIF.

SELECT vbeln, posnr, matnr, arktx, kwmeng, vrkme, netpr, waerk
FROM vbap
INTO TABLE @DATA(it_vbap)
FOR ALL ENTRIES IN @it_vbak
WHERE vbeln = @it_vbak-vbeln.

```

```

SELECT vbelv, posnv, vbeln, posnn
FROM vbfa
INTO TABLE @it_vbfa
FOR ALL ENTRIES IN @it_vbak
WHERE vbelv = @it_vbak-vbeln
      AND vbtyp_v = 'A' " Inquiry
      AND vbtyp_n = 'C'. " Sales Order

" Headers
LOOP AT it_vbak ASSIGNING FIELD-SYMBOL(<fs_vbak>).

      SELECT SINGLE vtext INTO @loc_vkorg_txt FROM tvkot " Sales
      Organization Description
      WHERE vkorg = @<fs_vbak>-vkorg AND spras = @sy-langu.

      SELECT SINGLE vtext INTO @loc_vtweg_txt FROM tvttw " Distribution
      Channel Description
      WHERE vtweg = @<fs_vbak>-vtweg AND spras = @sy-langu.

      SELECT SINGLE vtext INTO @loc_spart_txt FROM tspat " Division
      Description
      WHERE spart = @<fs_vbak>-spart AND spras = @sy-langu.

      SELECT SINGLE bezei INTO @loc_vkbur_txt FROM tvkbt " Sales Office
      Description
      WHERE vkbur = @<fs_vbak>-vkbur.

      SELECT SINGLE bezei INTO @loc_vkgrp_txt FROM tvgrt " Sales Group
      Description
      WHERE vkgrp = @<fs_vbak>-vkgrp.

DATA(wa_header) = VALUE zsd_rk_inquiry_header_s(
  wf_vbeln      = <fs_vbak>-vbeln
  wf_erdat      = <fs_vbak>-erdat
  wf_audat      = <fs_vbak>-audat
  wf_ernam      = <fs_vbak>-ernam
  wf_kunnr      = <fs_vbak>-kunnr
  wf_vkorg      = <fs_vbak>-vkorg
  wf_vtweg      = <fs_vbak>-vtweg
  wf_spart      = <fs_vbak>-spart
  wf_vkbur      = <fs_vbak>-vkbur
  wf_vkgrp      = <fs_vbak>-vkgrp
  wf_bstnk      = <fs_vbak>-bstnk
  wf_angdt      = <fs_vbak>-angdt
  wf_bnddt      = <fs_vbak>-bnddt
  wf_waerk      = <fs_vbak>-waerk
  wf_auart      = <fs_vbak>-auart
  wf_augru      = <fs_vbak>-augru
  wf_total_netwr = <fs_vbak>-netwr
  wf_vkorg_txt  = loc_vkorg_txt
  wf_vtweg_txt  = loc_vtweg_txt
  wf_spart_txt  = loc_spart_txt
  wf_vkbur_txt  = loc_vkbur_txt
  wf_vkgrp_txt  = loc_vkgrp_txt
).

loc_total_items = 0.
loc_total_netwr = 0.

```

```

loc_items_linked = 0.

CLEAR it_temp_items.

" Items
LOOP AT it_vbap ASSIGNING FIELD-SYMBOL(<fs_vbap>)
  WHERE vbeln = <fs_vbak>-vbeln.

  CLEAR: loc_vbeln_so, loc_posnr_so.

  loc_item_value = <fs_vbap>-kwmeng * <fs_vbap>-netpr.
  wg_is_ordered = ''.

  READ TABLE it_vbfa WITH KEY vbelv = <fs_vbap>-vbeln
                        posnv = <fs_vbap>-posnr
                        TRANSPORTING NO FIELDS.

  IF sy-subrc = 0.
    wg_is_ordered = 'X'.
    READ TABLE it_vbfa INTO wa_vbfa WITH KEY vbelv = <fs_vbap>-vbeln
posnv = <fs_vbap>-posnr.
    IF sy-subrc = 0.
      loc_vbeln_so = wa_vbfa-vbeln.
      loc_posnr_so = wa_vbfa-posnn.
    ENDIF.
  ENDIF.

  APPEND VALUE zsd_rk_inquiry_item_s(
    wf_vbeln      = <fs_vbap>-vbeln
    wf_posnr      = <fs_vbap>-posnr
    wf_matnr      = <fs_vbap>-matnr
    wf_arktx      = <fs_vbap>-arktx
    wf_kwmeng      = <fs_vbap>-kwmeng
    wf_vrkme      = <fs_vbap>-vrkme
    wf_netpr      = <fs_vbap>-netpr
    wf_netwr      = loc_item_value
    wf_waerk      = <fs_vbap>-waerk
    wg_is_ordered = wg_is_ordered
    wf_vbeln_so   = loc_vbeln_so
    wf_posnr_so   = loc_posnr_so
  ) TO it_temp_items.

  loc_total_items += 1.
  loc_total_netwr += loc_item_value.
  IF wg_is_ordered = 'X'.
    loc_items_linked += 1.
  ENDIF.
ENDLOOP.

wa_header-wf_total_items      = loc_total_items.
wa_header-wf_total_netwr     = loc_total_netwr.
wa_header-wf_ordered_items   = loc_items_linked.

IF loc_items_linked = 0.
  wa_header-wf_status_text = 'OPEN'.
ELSEIF loc_items_linked = loc_total_items.
  wa_header-wf_status_text = 'FULL'.
ELSE.
  wa_header-wf_status_text = 'PARTIAL'.

```

```

ENDIF.

wa_header-wg_fully_ordered      = COND #( WHEN loc_items_linked =
loc_total_items THEN 'X' ELSE '' ).
wa_header-wg_partially_ordered = COND #( WHEN loc_items_linked > 0 AND
loc_items_linked < loc_total_items THEN 'X' ELSE '' ).

IF iv_status IS INITIAL OR wa_header-wf_status_text = iv_status.
  APPEND wa_header TO et_inquiry_headers.
  APPEND LINES OF it_temp_items TO et_inquiry_items.
ENDIF.
ENDLOOP.

IF iv_status IS NOT INITIAL.
  ev_total_records = lines( et_inquiry_headers ).
  IF ev_total_records = 0.
    CLEAR et_inquiry_items.
    ev_message = 'No inquiries found for the specified status.'.
  ELSE.
    ev_message = 'Inquiries retrieved successfully'.
  ENDIF.
ELSE.
  ev_message = 'Inquiries retrieved successfully'.
ENDIF.

ENDFUNCTION.

```

Access URL:

http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_sd_rk_get_inquiry?sap-client=100

Sales Order

Access URL:

http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_sd_rk_get_cust_orders?sap-client=100

1. Create Data Dictionary Structures

Structure Name: **ZSD_RK_SALESORDER_HEADER_S**

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_VBELN	VBELN	VBAK	VBELN	Sales Order Number
WF_ERDAT	ERDAT	VBAK	ERDAT	Created On – date when order header was created
WF_AUDAT	AUDAT	VBAK	AUDAT	Document Date in Document – date of the sales order
WF_ERNAM	ERNAM	VBAK	ERNAM	Created By – user who created the sales order record
WF_KUNNR	KUNNR	VBAK	KUNNR	Customer Number – sold-to party for this sales order
WF_VKORG	VKORG	VBAK	VKORG	Sales Organization – responsible sales unit
WF_VTWEG	VTWEG	VBAK	VTWEG	Distribution Channel – channel through which items are sold
WF_SPART	SPART	VBAK	SPART	Division – business division for this sales order
WF_VKBUR	VKBUR	VBAK	VKBUR	Sales Office – office responsible for this sales order
WF_VKGRP	VKGRP	VBAK	VKGRP	Sales Group – group within the sales office
WF_BSTNK	BSTNK	VBAK	BSTNK	Customer Purchase Order Number (if provided)
WF_WAERK	WAERK	VBAK	WAERK	Currency Key – currency in which order net values are stored
WF_AUART	AUART	VBAK	AUART	Sales Document Type – for this function, always “TA” (Standard Order)

WF_NETWR	NETWR	VBAK	NETWR	Net Order Value – total net value from the header record
WF_VKORG_TXT	VTEXT	TVKOT	VTEXT	Sales Org. Description – language-specific text for VKORG
WF_VTWEG_TXT	VTEXT	TVTWT	VTEXT	Distribution Channel Description – language-specific text for VTWEG
WF_SPART_TXT	VTEXT	TSPAT	VTEXT	Division Description – language-specific text for SPART
WF_VKBUR_TXT	BEZEI	TVKBT	BEZEI	Sales Office Description – text for VKBUR
WF_VKGRP_TXT	BEZEI	TVGRT	BEZEI	Sales Group Description – text for VKGRP
WF_TOTAL_ITEMS	I	– (calculated)	–	Total Number of Line Items in this order
WF_TOTAL_NETWR	P (DEC15_2)	– (calculated)	–	Total Net Value – sum of (quantity × price) for all items
WF_DELIVERED_ITEMS	I	– (calculated)	–	Number of Items That Have Been Delivered
WF_STATUS_TEXT	CHAR10	– (calculated)	–	Order Status – “OPEN” / “PARTIAL” / “FULL” (based on WF_DELIVERED_ITEMS)
WG_FULLY_DELIVERED	CHAR1	– (calculated)	–	Fully Delivered Flag – “X” if all items delivered
WG_PARTIALLY_DELIVERED	CHAR1	– (calculated)	–	Partially Delivered Flag – “X” if some (but not all) items delivered

1.2 Item Structure: ZSD_RK_SALESORDER_ITEM_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_VBELN	VBELN	VBAP	VBELN	Sales Order Number (header reference)
WF_POSNR	POSNR	VBAP	POSNR	Item Number – position in the order

WF_MATNR	MATNR	VBAP	MATNR	Material Number – material being ordered
WF_ARKTX	ARKTX	VBAP	ARKTX	Short Text – item description (free-text)
WF_KWMENG	KWMENG	VBAP	KWMENG	Order Quantity – quantity ordered in base unit
WF_VRKME	VRKME	VBAP	VRKME	Sales Unit – unit of measure for the ordered quantity
WF_NETPR	NETPR	VBAP	NETPR	Net Price – price per sales unit
WF_NETWR	P (DEC15_2)	– (calculated)	–	Line Net Value – calculated as KWMENG × NETPR
WF_WAERK	WAERK	VBAP	WAERK	Currency Key – currency in which NETPR is given
WG_IS_DELIVERED	CHAR1	– (calculated)	–	Delivered Flag – “X” if this line has at least one outbound delivery
WF_VBELN_DEL	VBELN	VBFA (calculated)	VBELN (VBFA-VBELN)	Delivery Document Number – if this line was delivered
WF_POSNR_DEL	POSNR	VBFA (calculated)	POSNN	Delivery Item Number – position of the delivered line in the delivery

2. Create Table Types

- **ZSD_RK_SALESORDER_HEADER_T** — Table type for **ZSD_RK_SALESORDER_ITEM_S**.
- **ZSD_RK_SALESORDER_ITEM_T** — Table type for **ZSD_RK_SALESORDER_ITEM_S**.

3. Create Function Module: **zfm_sd_rk_get_cust_order**

```
FUNCTION zfm_sd_rk_get_cust_order.
```

```

*-----
*""Local Interface:
*  IMPORTING
*      VALUE (IV_KUNNR) TYPE  KUNNR
*      VALUE (IV_DATE_FROM) TYPE  ERDAT DEFAULT 00000000
*      VALUE (IV_DATE_TO) TYPE  ERDAT DEFAULT 99991231
*      VALUE (IV_SORT_NEWEST) TYPE  STRING OPTIONAL
*      VALUE (IV_PAGE_NUM) TYPE  I DEFAULT 1
*      VALUE (IV_PAGE_SIZE) TYPE  I DEFAULT 10

```



```

*"      VALUE(IV_VBELN) TYPE  VBELN_VA OPTIONAL
*"      VALUE(IV_STATUS) TYPE  STRING OPTIONAL
*"      EXPORTING
*"      VALUE(ET_SALESORDER_HEADERS) TYPE  ZSD_RK_SALESORDER_HEADER_T
*"      VALUE(ET_SALESORDER_ITEMS) TYPE  ZSD_RK_SALESORDER_ITEM_T
*"      VALUE(EV_TOTAL_RECORDS) TYPE  I
*"      VALUE(EV_MESSAGE) TYPE  STRING
*"-----

" Input Validation
IF iv_kunnr IS INITIAL.
    ev_message = 'Customer number is required'.
    RETURN.
ENDIF.

TYPES: BEGIN OF ty_vbfa_partial,
        vbelv TYPE vbfa-vbelv,
        posnv TYPE vbfa-posnv,
        vbeln TYPE vbfa-vbeln,
        posnn TYPE vbfa-posnn,
    END OF ty_vbfa_partial.

DATA: it_vbfa TYPE TABLE OF ty_vbfa_partial,
      wa_vbfa TYPE ty_vbfa_partial.

" Variables Declaration for Fetching text values separately
DATA: loc_vkorg_txt TYPE tvkot-vtext,
      loc_vtweg_txt TYPE tvttwt-vtext,
      loc_spart_txt TYPE tspat-vtext,
      loc_vkbur_txt TYPE tvkbt-bezei,
      loc_vkgrp_txt TYPE tvkgt-bezei.

" Variables Declaration for calculating total items, netValue, Delivered Items
DATA: loc_total_items      TYPE i VALUE 0,
      loc_total_netwr      TYPE netwr VALUE 0,
      loc_items_delivered  TYPE i VALUE 0,
      loc_item_value       TYPE p LENGTH 15 DECIMALS 2.

" Variable Flag to store If order item is delivered
DATA: wg_is_delivered TYPE char1.

" temporary table to hold items per header
DATA: lt_temp_items TYPE zsd_rk_salesorder_item_t.

" Variables to store the delivered items document number and Item document number
DATA: loc_vbeln_del TYPE vbeln,
      loc_posnr_del TYPE posnr.

" Internal Table for storing the Header items
DATA: it_vbak TYPE STANDARD TABLE OF vbak WITH EMPTY KEY.

" Set Default Date Range
DATA(loc_date_from) = COND erdat( WHEN iv_date_from IS INITIAL THEN '00000000' ELSE
iv_date_from ).
DATA(loc_date_to)    = COND erdat( WHEN iv_date_to IS INITIAL THEN '99991231' ELSE
iv_date_to ).

" Calculate Pagination Offset

```

```

DATA(loc_offset) = ( iv_page_num - 1 ) * iv_page_size.

" Main Data Selection with Pagination and Sorting
IF iv_vbeln IS NOT INITIAL.
    " If VBELN is provided, fetch specific inquiry
    SELECT vbeln, erdat, audat, ernam, kunnr, vkorg, vtweg, spart,
           vkbur, vkgrp, bstnk, angdt, bnddt, waerk, auart, augru, netwr
    FROM vbak
    INTO CORRESPONDING FIELDS OF TABLE @it_vbak
    WHERE vbeln = @iv_vbeln
           AND auart = 'TA'.
ELSEIF iv_status IS INITIAL.
    " Only apply pagination and sorting if status is not requested
    IF iv_sort_newest = 'X'.
        SELECT vbeln, erdat, audat, ernam, kunnr, vkorg, vtweg, spart,
               vkbur, vkgrp, bstnk, angdt, bnddt, waerk, auart, augru, netwr
        FROM vbak
        WHERE auart = 'TA'
               AND kunnr = @iv_kunnr
               AND audat BETWEEN @loc_date_from AND @loc_date_to
        ORDER BY audat DESCENDING
        INTO CORRESPONDING FIELDS OF TABLE @it_vbak
        UP TO @iv_page_size ROWS
        OFFSET @loc_offset.
    ELSE.
        SELECT vbeln, erdat, audat, ernam, kunnr, vkorg, vtweg, spart,
               vkbur, vkgrp, bstnk, angdt, bnddt, waerk, auart, augru, netwr
        FROM vbak
        WHERE auart = 'TA'
               AND kunnr = @iv_kunnr
               AND audat BETWEEN @loc_date_from AND @loc_date_to
        ORDER BY audat ASCENDING
        INTO CORRESPONDING FIELDS OF TABLE @it_vbak
        UP TO @iv_page_size ROWS
        OFFSET @loc_offset.
    ENDIF.
ELSE.
    " Status is provided, so no pagination or sorting - fetch full data set
    IF iv_sort_newest = 'X'.
        SELECT vbeln, erdat, audat, ernam, kunnr, vkorg, vtweg, spart,
               vkbur, vkgrp, bstnk, angdt, bnddt, waerk, auart, augru, netwr
        FROM vbak
        INTO CORRESPONDING FIELDS OF TABLE @it_vbak
        WHERE auart = 'TA'
               AND kunnr = @iv_kunnr
               AND audat BETWEEN @loc_date_from AND @loc_date_to
        ORDER BY audat DESCENDING.
    ELSE.
        SELECT vbeln, erdat, audat, ernam, kunnr, vkorg, vtweg, spart,
               vkbur, vkgrp, bstnk, angdt, bnddt, waerk, auart, augru, netwr
        FROM vbak
        INTO CORRESPONDING FIELDS OF TABLE @it_vbak
        WHERE auart = 'TA'
               AND kunnr = @iv_kunnr
               AND audat BETWEEN @loc_date_from AND @loc_date_to
        ORDER BY audat ASCENDING.
    ENDIF.
ENDIF.

```

```

" Exit if No Data Found
IF it_vbak IS INITIAL.
    ev_message = 'No sales orders found for specified criteria'.
    ev_total_records = 0.
    RETURN.
ENDIF.

" Calculate Total Records if paginated else ignore
IF iv_vbeln IS NOT INITIAL.
    ev_total_records = lines( it_vbak ).

ELSEIF iv_status IS INITIAL.
    " Count for pagination
    SELECT COUNT(*) INTO @ev_total_records
    FROM vbak
    WHERE auart = 'TA'
    AND kunnr = @iv_kunnr
    AND audat BETWEEN @loc_date_from AND @loc_date_to.

ELSE.
    " Status-based filtering - count only after loop
    ev_total_records = lines( et_salesorder_headers ).

ENDIF.

" Fetch Related Items
SELECT vbeln, posnr, matnr, arktx, kwmeng, vrkme, netpr, waerk, charg
FROM vbap
INTO TABLE @DATA(it_vbap)
FOR ALL ENTRIES IN @it_vbak
WHERE vbeln = @it_vbak-vbeln.

" Check which items were delivered
SELECT vbelv, posnv, vbeln, posnn
FROM vbfa
INTO TABLE @it_vbfa
FOR ALL ENTRIES IN @it_vbak
WHERE vbelv = @it_vbak-vbeln
    AND vbtyp_v = 'C' " Sales Order
    AND vbtyp_n = 'J'. " Delivery

" Process Headers
LOOP AT it_vbak ASSIGNING FIELD-SYMBOL(<fs_vbak>).

    SELECT SINGLE vtext INTO @loc_vkorg_txt FROM tvkot " Sales Organization
    Description
    WHERE vkorg = @<fs_vbak>-vkorg AND spras = @sy-langu.

    SELECT SINGLE vtext INTO @loc_vtweg_txt FROM tvttw " Distribution Channel
    Description
    WHERE vtweg = @<fs_vbak>-vtweg AND spras = @sy-langu.

    SELECT SINGLE vtext INTO @loc_spart_txt FROM tspat " Division Description
    WHERE spart = @<fs_vbak>-spart AND spras = @sy-langu.

    SELECT SINGLE bezei INTO @loc_vkbur_txt FROM tvkbt " Sales Office Description
    WHERE vkbur = @<fs_vbak>-vkbur.

```

```
SELECT SINGLE bezei INTO @loc_vkgrp_txt FROM tvgrt
WHERE vkgrp = @<fs_vbak>-vkgrp.
```

" Sales Group Description

" Creating Header Structure

```
DATA(wa_header) = VALUE zsd_rk_salesorder_header_s(
  wf_vbeln      = <fs_vbak>-vbeln
  wf_erdat      = <fs_vbak>-erdat
  wf_audat      = <fs_vbak>-audat
  wf_ernam      = <fs_vbak>-ernam
  wf_kunnr      = <fs_vbak>-kunnr
  wf_vkorg      = <fs_vbak>-vkorg
  wf_vtweg      = <fs_vbak>-vtweg
  wf_spart      = <fs_vbak>-spart
  wf_vkbur      = <fs_vbak>-vkbur
  wf_vkgrp      = <fs_vbak>-vkgrp
  wf_bstnk      = <fs_vbak>-bstnk
  wf_waerk      = <fs_vbak>-waerk
  wf_auart      = <fs_vbak>-auart
  wf_netwr      = <fs_vbak>-netwr
  wf_vkorg_txt  = loc_vkorg_txt
  wf_vtweg_txt  = loc_vtweg_txt
  wf_spart_txt  = loc_spart_txt
  wf_vkbur_txt  = loc_vkbur_txt
  wf_vkgrp_txt  = loc_vkgrp_txt
).
```

```
loc_total_items = 0.
loc_total_netwr = 0.
loc_items_delivered = 0.
```

" Reset item collector

```
CLEAR lt_temp_items.
```

" Loop through items

```
LOOP AT it_vbap ASSIGNING FIELD-SYMBOL(<fs_vbap>)
WHERE vbeln = <fs_vbak>-vbeln.
```

" Reset delivery refs

```
CLEAR: loc_vbeln_del, loc_posnr_del.
```

" Calculate item value

```
loc_item_value = <fs_vbap>-kwmeng * <fs_vbap>-netpr.
wg_is_delivered = ''.
```

" Check delivery

```
READ TABLE it_vbfa WITH KEY vbelv = <fs_vbap>-vbeln
                                posnv = <fs_vbap>-posnr
                                TRANSPORTING NO FIELDS.
```

```
IF sy-subrc = 0.
```

```
  wg_is_delivered = 'X'.
```

```
  READ TABLE it_vbfa INTO wa_vbfa WITH KEY vbelv = <fs_vbap>-vbeln posnv =
<fs_vbap>-posnr.
```

```
  IF sy-subrc = 0.
```

```
    loc_vbeln_del = wa_vbfa-vbeln.
```

```
    loc_posnr_del = wa_vbfa-posnr.
```

```
  ENDIF.
```

```
ENDIF.
```

```

" Append to temp item list
APPEND VALUE zsd_rk_salesorder_item_s(
  wf_vbeln      = <fs_vbap>-vbeln
  wf_posnr      = <fs_vbap>-posnr
  wf_matnr      = <fs_vbap>-matnr
  wf_arktx      = <fs_vbap>-arktx
  wf_kwmeng     = <fs_vbap>-kwmeng
  wf_vrkme      = <fs_vbap>-vrkme
  wf_netpr      = <fs_vbap>-netpr
  wf_netwr      = loc_item_value
  wf_waerk      = <fs_vbap>-waerk
  wg_is_delivered = wg_is_delivered
  wf_vbeln_del   = loc_vbeln_del
  wf_posnr_del   = loc_posnr_del
) TO lt_temp_items.

" Totals
loc_total_items += 1.
loc_total_netwr += loc_item_value.
IF wg_is_delivered = 'X'.
  loc_items_delivered += 1.
ENDIF.

ENDLOOP.

" Finalize header and apply status filter
wa_header-wf_total_items      = loc_total_items.
wa_header-wf_total_netwr     = loc_total_netwr.
wa_header-wf_delivered_items  = loc_items_delivered.

IF loc_items_delivered = 0.
  wa_header-wf_status_text = 'OPEN'.
ELSEIF loc_items_delivered = loc_total_items.
  wa_header-wf_status_text = 'FULL'.
ELSE.
  wa_header-wf_status_text = 'PARTIAL'.
ENDIF.

wa_header-wg_fully_delivered      = COND #( WHEN loc_items_delivered = loc_total_items
THEN 'X' ELSE '' ).
wa_header-wg_partially_delivered = COND #( WHEN loc_items_delivered > 0 AND
loc_items_delivered < loc_total_items THEN 'X' ELSE '' ).

" Apply filter - add items only if status matches
IF iv_status IS INITIAL OR wa_header-wf_status_text = iv_status.
  APPEND wa_header TO et_salesorder_headers.
  APPEND LINES OF lt_temp_items TO et_salesorder_items.
ENDIF.

ENDLOOP.

" Return success message
IF iv_status IS NOT INITIAL.
  ev_total_records = lines( et_salesorder_headers ).
  IF ev_total_records = 0.
    CLEAR et_salesorder_items.
    ev_message = 'No sales orders found for the specified status.'.
  
```

```
ELSE.  
    ev_message = 'Sales orders retrieved successfully'.  
ENDIF.  
ELSE.  
    ev_message = 'Sales orders retrieved successfully'.  
ENDIF.  
  
ENDFUNCTION.
```

Delivery

Access URL:

http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_sd_rk_get_cust_deliveries?sap-client=100

Structure Name: ZSD_RK_DELIVERY_HEADER_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_VBELN	VBELN_VL	LIKP	VBELN	Delivery Number – Sales document number (delivery)
WF_LFDAT	LFDAT	LIKP	LFDAT	Delivery Date – Requested delivery date
WF_WADAT	WADAT_IST	LIKP	WADAT_IST	Actual GI Date – Goods issue posting date
WF_ERDAT	ERDAT	LIKP	ERDAT	Created On – Date the delivery was created
WF_VSTEL	VSTEL	LIKP	VSTEL	Shipping Point – Point of departure
WF_VKORG	VKORG	LIKP	VKORG	Sales Organization – Sales org responsible
WF_VTWIV	VTWEG	LIKP	VTWEG	Distribution Channel – Channel through which goods are distributed
WF_LFART	LFART	LIKP	LFART	Delivery Type – Type of delivery (standard, returns, etc.)
WF_KUNAG	KUNAG	LIKP	KUNAG	Sold-To Party – Customer placing the order
WF_KUNNR	KUNNR	LIKP	KUNNR	Ship-To Party – Delivery recipient
WF_FKDAT	FKDAT	LIKP	FKDAT	Billing Date – Invoice date
WF_LDDAT	LDDAT	LIKP	LDDAT	Loading Date – Planned loading date
WF_TDDAT	TDDAT	LIKP	TDDAT	Transportation Planning Date – Planned shipping date
WF_VSBED	VSBED	LIKP	VSBED	Shipping Conditions – Priority/rule for shipping
WF_ROUTE	ROUTE	LIKP	ROUTE	Route – Shipping route
WF_LIFSK	LIFSK	LIKP	LIFSK	Delivery Block – Block for delivery creation
WF_FAKSK	FAKSK	LIKP	FAKSK	Billing Block – Block for billing
WF_VKBUR	VKBUR	LIKP	VKBUR	Sales Office – Responsible sales office
WF_WAERK	WAERK	LIKP	WAERK	Currency – Transaction currency
WF_VSTEL_TXT	VTEXT	TVSTT	VTEXT	Shipping Point Description

WF_VKORG_TXT	VTEXT	TVKOT	VTEXT	Sales Organization Description
WF_VTWIV_TXT	VTEXT	TVTWT	VTEXT	Distribution Channel Description
WF_LFART_TXT	VTEXT	TVLKT	VTEXT	Delivery Type Description
WF_VSBED_TXT	VTEXT	TVSBT	VTEXT	Shipping Condition Description
WF_FAKSK_TXT	VTEXT	TVFST	VTEXT	Billing Block Description
WF_VKBUR_TXT	BEZEI		BEZEI	Sales Office Description
WF_TOTAL_ITEMS	INT4	—	—	Number of delivery items (computed)
WF_TOTAL_NETWR	NETWR	—	—	Total Net Value – Computed from items
WF_BILLED_ITEMS	INT4	—	—	Number of items billed
WF_PICKED_ITEMS	INT4	—	—	Number of items picked
WF_PACKED_ITEMS	INT4	—	—	Number of items packed
WF_STATUS_TEXT	CHAR10	—	—	Delivery status text (OPEN / PARTIAL / FULL)
WG_FULLY_BILLED	FLAG	—	—	Fully Billed Flag – 'X' if all items billed
WG_PARTIALLY_BILLED	FLAG	—	—	Partially Billed Flag – 'X' if some items billed

◆ 2. Item-Level Structure

Structure Name: ZSD_RK_DELIVERY_ITEM_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_VBELN	VBELN_VL	LIPS	VBELN	Delivery Number
WF_POSNR	POSNR_VL	LIPS	POSNR	Delivery Item Number
WF_MATNR	MATNR	LIPS	MATNR	Material Number
WF_ARKTX	ARKTX	LIPS	ARKTX	Material Description
WF_LFIMG	LFIMG	LIPS	LFIMG	Delivery Quantity
WF_NAME2	NAME2	T001W	NAME2	Plant Name
WF_STRAS	STRAS	T001W	STRAS	Street
WF_PSTLZ	PSTLZ	T001W	PSTLZ	Postal Code
WF_ORT01	ORT01	T001W	ORT01	City
WF_LGOBE	LGOBE	T001L	LGOBE	Storage Location Description
WF_MEINS	MEINS	LIPS	MEINS	Base Unit of Measure
WF_NETWR	NETWR	— (calc)	—	Net Value – Calculated: LFIMG × item price

WF_WERKS	WERKS_D	LIPS	WERKS	Plant
WF_VGBEL	VGBEL	LIPS	VGBEL	Preceding Sales Document (SO)
WF_VGPOS	VGPOS	LIPS	VGPOS	Preceding Sales Item
WF_CHARG	CHARG_D	LIPS	CHARG	Batch Number
WF_LGMNG	LGMNG	LIPS	LGMNG	Quantity in WM
WG_IS_BILLED	FLAG	VBFA (calc)	—	Is Billed Flag – 'X' if billing doc exists
WF_VBELN_BILL	VBELN	VBFA (calc)	VBELV	Billing Document Number
WF_POSNR_BILL	POSNR	VBFA (calc)	POSNV	Billing Document Item
WG_PICKED	KOSTA	VBUP	KOSTA	Picking Status
WG_PACKED	PKSTA	VBUP	PKSTA	Packing Status
WF_GI_STATUS	WBSTA	VBUP	WBSTA	Goods Issue Status

3. Table Types

Table Type Name	Line Type Structure
ZSD_RK_DELIVERY_HEADER_T	ZSD_RK_DELIVERY_HEADER_S
ZSD_RK_DELIVERY_ITEM_T	ZSD_RK_DELIVERY_ITEM_S

ABAP Code for the Function Module

```

FUNCTION ZFM_SD_RK_GET_CUST_DELIVERY.
* "-----
* " "Local Interface:
* "  IMPORTING
* "    VALUE(IV_KUNAG) TYPE KUNAG
* "    VALUE(IV_DATE_FROM) TYPE LFDAT DEFAULT 00000000
* "    VALUE(IV_DATE_TO) TYPE LFDAT DEFAULT 99991231
* "    VALUE(IV_SORT_NEWEST) TYPE STRING OPTIONAL
* "    VALUE(IV_PAGE_NUM) TYPE I DEFAULT 1
* "    VALUE(IV_PAGE_SIZE) TYPE I DEFAULT 10
* "    VALUE(IV_VBELN) TYPE VBELN_VL OPTIONAL
* "    VALUE(IV_STATUS) TYPE STRING OPTIONAL
* "  EXPORTING
* "    VALUE(ET_DELIVERY_HEADERS) TYPE ZSD_RK_DELIVERY_HEADER_T
* "    VALUE(ET_DELIVERY_ITEMS) TYPE ZSD_RK_DELIVERY_ITEM_T
* "    VALUE(EV_TOTAL_RECORDS) TYPE I
* "    VALUE(EV_MESSAGE) TYPE STRING
* "-----

IF iv_kunag IS INITIAL.
  ev_message = 'Customer number is required'.
  RETURN.
ENDIF.

TYPES: BEGIN OF ty_vbfa_partial,
        vbelv TYPE vbfa-vbelv,

```

```

        posnv TYPE vbfa-posnv,
        vbeln TYPE vbfa-vbeln,
        posnn TYPE vbfa-posnn,
    END OF ty_vbfa_partial.

DATA: it_vbfa TYPE TABLE OF ty_vbfa_partial,
      wa_vbfa TYPE ty_vbfa_partial.

DATA:
    loc_vstel_txt TYPE tvstt-vtext,
    loc_vkorg_txt TYPE tvkot-vtext,
    loc_vtwiv_txt TYPE tvttwt-vtext,
    loc_lfart_txt TYPE tvlkt-vtext,
    loc_vsbed_txt TYPE tvsbt-vtext,
    loc_faksp_txt TYPE tvfst-vtext,
    loc_vkbur_txt TYPE tvkbt-bezei.

DATA:
    loc_item_value    TYPE i VALUE 0,
    loc_total_items   TYPE i,
    loc_total_netwr    TYPE netwr,
    loc_items_billed   TYPE i,
    loc_items_picked   TYPE i,
    loc_items_packed   TYPE i.

DATA: wg_is_delivered TYPE char1,
      loc_vbeln_bill   TYPE vbeln_von,
      loc_posnr_bill   TYPE posnr_von.

DATA:
    it_likp          TYPE TABLE OF likp,
    it_temp_item TYPE zsd_rk_delivery_item_t.

DATA(loc_date_from) = COND lfdat( WHEN iv_date_from IS INITIAL THEN
'00000000' ELSE iv_date_from ).
DATA(loc_date_to)    = COND lfdat( WHEN iv_date_to   IS INITIAL THEN
'99991231' ELSE iv_date_to ).

DATA(loc_offset) = ( iv_page_num - 1 ) * iv_page_size.

IF iv_vbeln IS NOT INITIAL.
    SELECT * FROM likp
        INTO TABLE @it_likp
        WHERE vbeln = @iv_vbeln
            AND kunag = @iv_kunag.

ELSEIF iv_status IS INITIAL.
    IF iv_sort_newest = 'X'.
        SELECT * FROM likp
            WHERE kunag = @iv_kunag
                AND lfdat BETWEEN @loc_date_from AND @loc_date_to
            ORDER BY lfdat DESCENDING
            INTO TABLE @it_likp
            UP TO @iv_page_size ROWS
            OFFSET @loc_offset.
    ELSE.
        SELECT * FROM likp
            WHERE kunag = @iv_kunag

```

```

        AND lfdat BETWEEN @loc_date_from AND @loc_date_to
ORDER BY lfdat ASCENDING
INTO TABLE @it_likp
UP TO @iv_page_size ROWS
OFFSET @loc_offset.
ENDIF.
ELSE.
IF iv_sort_newest = 'X'.
SELECT * FROM likp
INTO TABLE @it_likp
WHERE kunag = @iv_kunag
AND lfdat BETWEEN @loc_date_from AND @loc_date_to
ORDER BY lfdat DESCENDING.
ELSE.
SELECT * FROM likp
INTO TABLE @it_likp
WHERE kunag = @iv_kunag
AND lfdat BETWEEN @loc_date_from AND @loc_date_to
ORDER BY lfdat ASCENDING.
ENDIF.
ENDIF.

IF it_likp IS INITIAL.
ev_message = 'No deliveries found for specified criteria'.
ev_total_records = 0.
RETURN.
ENDIF.

IF iv_vbeln IS NOT INITIAL.
ev_total_records = lines( it_likp ).
ELSEIF iv_status IS INITIAL.
SELECT COUNT(*) FROM likp
INTO @ev_total_records
WHERE kunag = @iv_kunag
AND lfdat BETWEEN @loc_date_from AND @loc_date_to.
ENDIF.

SELECT * FROM lips
INTO TABLE @DATA(it_lips)
FOR ALL ENTRIES IN @it_likp
WHERE vbeln = @it_likp-vbeln.

SELECT vbelv, posnv, vbeln, posnn
FROM vbfa
INTO TABLE @it_vbfa
FOR ALL ENTRIES IN @it_likp
WHERE vbelv = @it_likp-vbeln
AND vbtyp_v = 'J'
AND vbtyp_n = 'M'.

SELECT * FROM vbup
INTO TABLE @DATA(it_vbup)
FOR ALL ENTRIES IN @it_likp
WHERE vbeln = @it_likp-vbeln.

SELECT vbeln, posnr, netwr, kwmeng
FROM vbap
INTO TABLE @DATA(it_vbap)

```

```

FOR ALL ENTRIES IN @it_lips
WHERE vbeln = @it_lips-vgbel
      AND posnr = @it_lips-vgpos.

LOOP AT it_likp ASSIGNING FIELD-SYMBOL(<fs_likp>).
  CLEAR:
    loc_vstel_txt, loc_vkorg_txt, loc_vtwiv_txt,
    loc_lfart_txt, loc_vsbed_txt, loc_faksp_txt,
    loc_vkbur_txt,
    loc_total_items, loc_total_netwr,
    loc_items_billed, loc_items_picked, loc_items_packed.

  SELECT SINGLE vtext INTO @loc_vstel_txt FROM tvstt
    WHERE vstel = @<fs_likp>-vstel AND spras = @sy-langu.
  SELECT SINGLE vtext INTO @loc_vkorg_txt FROM tvkot
    WHERE vkorg = @<fs_likp>-vkorg AND spras = @sy-langu.
  SELECT SINGLE vtext INTO @loc_vtwiv_txt FROM tvttw
    WHERE vtweg = @<fs_likp>-vtwiv AND spras = @sy-langu.
  SELECT SINGLE vtext INTO @loc_lfart_txt FROM tvlkt
    WHERE lfart = @<fs_likp>-lfart AND spras = @sy-langu.
  SELECT SINGLE vtext INTO @loc_vsbed_txt FROM tvsbt
    WHERE vsbed = @<fs_likp>-vsbed AND spras = @sy-langu.
  SELECT SINGLE vtext INTO @loc_faksp_txt FROM tvfst
    WHERE faksp = @<fs_likp>-faksk AND spras = @sy-langu.
  SELECT SINGLE bezei INTO @loc_vkbur_txt FROM tvkbt
    WHERE vkbur = @<fs_likp>-vkbur.

  DATA(wa_header) = VALUE zsd_rk_delivery_header_s(
    wf_vbeln      = <fs_likp>-vbeln
    wf_lfdat      = <fs_likp>-lfdat
    wf_wadat      = <fs_likp>-wadat
    wf_erdat      = <fs_likp>-erdat
    wf_vstel      = <fs_likp>-vstel
    wf_vkorg      = <fs_likp>-vkorg
    wf_vtwiv      = <fs_likp>-vtwiv
    wf_lfart      = <fs_likp>-lfart
    wf_kunag      = <fs_likp>-kunag
    wf_kunnr      = <fs_likp>-kunnr
    wf_fkdat      = <fs_likp>-fkdat
    wf_lddat      = <fs_likp>-lddat
    wf_tddat      = <fs_likp>-tddat
    wf_vsbed      = <fs_likp>-vsbed
    wf_route      = <fs_likp>-route
    wf_lifsk      = <fs_likp>-lifsk
    wf_faksk      = <fs_likp>-faksk
    wf_vkbur      = <fs_likp>-vkbur
    wf_waerk      = <fs_likp>-waerk
    wf_vstel_txt  = loc_vstel_txt
    wf_vkorg_txt  = loc_vkorg_txt
    wf_vtwiv_txt  = loc_vtwiv_txt
    wf_lfart_txt  = loc_lfart_txt
    wf_vsbed_txt  = loc_vsbed_txt
    wf_faksk_txt  = loc_faksp_txt
    wf_vkbur_txt  = loc_vkbur_txt
  ).

  CLEAR it_temp_item.

```

```

    LOOP AT it_lips ASSIGNING FIELD-SYMBOL(<fs_lips>) WHERE vbeln =
<fs_lipk>-vbeln.

        CLEAR: loc_vbeln_bill, loc_posnr_bill, loc_item_value, wg_is_delivered.

        READ TABLE it_vbup INTO DATA(wa_vbup)
            WITH KEY vbeln = <fs_lips>-vbeln posnr = <fs_lips>-posnr.

        READ TABLE it_vbfa WITH KEY vbelv = <fs_lips>-vbeln
            posnv = <fs_lips>-posnr
            TRANSPORTING NO FIELDS.

        IF sy-subrc = 0.
            wg_is_delivered = 'X'.
            READ TABLE it_vbfa INTO wa_vbfa WITH KEY vbelv = <fs_lips>-vbeln
posnv = <fs_lips>-posnr.
            IF sy-subrc = 0.
                loc_vbeln_bill = wa_vbfa-vbeln.
                loc_posnr_bill = wa_vbfa-posnr.
            ENDIF.
        ENDIF.

        READ TABLE it_vbap INTO DATA(wa_vbap)
            WITH KEY vbeln = <fs_lips>-vgbel
            posnr = <fs_lips>-vgpos.
        IF sy-subrc = 0 AND wa_vbap-kwmeng > 0.
            loc_item_value = ( wa_vbap-netwr / wa_vbap-kwmeng ) *
<fs_lips>-lfimg.
        ENDIF.

        SELECT SINGLE name2, stras, pstlz, ort01, lgobe
        FROM t001w
        LEFT JOIN t001l ON t001w~werks = t001l~werks
            AND t001l~lgort = @<fs_lips>-lgort
        INTO @DATA(wa_plant)
        WHERE t001w~werks = @<fs_lips>-werks.

        APPEND VALUE #(
            wf_vbeln      = <fs_lips>-vbeln
            wf_posnr      = <fs_lips>-posnr
            wf_matnr      = <fs_lips>-matnr
            wf_arktx      = <fs_lips>-arktx
            wf_lfimg      = <fs_lips>-lfimg
            wf_name2      = wa_plant-name2
            wf_stras      = wa_plant-stras
            wf_pstlz      = wa_plant-pstlz
            wf_ort01      = wa_plant-ort01
            wf_lgobe      = wa_plant-lgobe
            wf_meins      = <fs_lips>-meins
            wf_netwr      = loc_item_value
            wf_werks      = <fs_lips>-werks
            wf_vgbel      = <fs_lips>-vgbel
            wf_vgpos      = <fs_lips>-vgpos
            wf_charg      = <fs_lips>-charg
            wf_lgmng      = <fs_lips>-lgmng
            wg_is_billed  = wg_is_delivered
            wf_vbeln_bill = loc_vbeln_bill
            wf_posnr_bill = loc_posnr_bill

```

```

        wg_picked          = COND #( WHEN wa_vbup IS NOT INITIAL THEN
wa_vbup-kosta ELSE '' )
        wg_packed          = COND #( WHEN wa_vbup IS NOT INITIAL THEN
wa_vbup-pksta ELSE '' )
        wf_gi_status       = COND #( WHEN wa_vbup IS NOT INITIAL THEN
wa_vbup-wbsta ELSE '' )
    ) TO it_temp_item.

    loc_total_items  += 1.
    loc_total_netwr  += loc_item_value.
    IF wg_is_delivered = 'X'.
        loc_items_billed += 1.
    ENDIF.
    IF wa_vbup-kosta = 'C'.
        loc_items_picked += 1.
    ENDIF.
    IF wa_vbup-pksta = 'C'.
        loc_items_packed += 1.
    ENDIF.
ENDLOOP.

wa_header-wf_total_items      = loc_total_items.
wa_header-wf_total_netwr      = loc_total_netwr.
wa_header-wf_billed_items     = loc_items_billed.
wa_header-wf_picked_items     = loc_items_picked.
wa_header-wf_packed_items     = loc_items_packed.

IF loc_items_billed = loc_total_items.
    wa_header-wf_status_text = 'FULL'.
    wa_header-wg_fully_billed = 'X'.
ELSEIF loc_items_billed > 0.
    wa_header-wf_status_text = 'PARTIAL'.
    wa_header-wg_partially_billed = 'X'.
ELSE.
    wa_header-wf_status_text = 'OPEN'.
    CLEAR: wa_header-wg_fully_billed, wa_header-wg_partially_billed.
ENDIF.

IF iv_status IS INITIAL OR wa_header-wf_status_text = iv_status.
    APPEND wa_header TO et_delivery_headers.
    APPEND LINES OF it_temp_item TO et_delivery_items.
ENDIF.
ENDLOOP.

IF iv_status IS NOT INITIAL.
    ev_total_records = lines( et_delivery_headers ).
    IF ev_total_records = 0.
        CLEAR et_delivery_items.
        ev_message = 'No sales orders found for the specified status.'.
    ELSE.
        ev_message = 'Sales orders retrieved successfully'.
    ENDIF.
ELSE.
    ev_message = 'Sales orders retrieved successfully'.
ENDIF.

ENDFUNCTION.

```

DASHBOARD

Access URL:

http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_sd_rk_get_cust_dashboard?sap-client=100

Structure Name: ZSD_RK_CUST_DASH_KPI_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_AVG_DELIVERY_DAYS	I	calculated	—	Average Delivery Days (Total delivery days ÷ count of delivered docs)
WF_INQUIRY_COUNT	I	calculated	—	Total number of inquiries (rows in VBAK where AUART = 'AF')
WF_INQUIRY_NET_VAL	NETWR	VBAK	NETWR	Sum of net values for inquiries (VBAK-NETWR)
WF_SO_COUNT	I	calculated	—	Total number of sales orders (rows in VBAK where AUART = 'TA')
WF_SO_NET_VAL	NETWR	VBAK	NETWR	Sum of net values for sales orders (VBAK-NETWR)
WF_DEL_COUNT	I	calculated	—	Total number of deliveries (rows in LIKP)
WF_INQUIRY_OPEN_CNT	I	calculated	—	Count of inquiries with 0 VBFA references (open inquiries)
WF_INQUIRY_PARTIAL_CONV	I	calculated	—	Count of inquiries with VBFA refs < item count (partial conversion)
WF_INQUIRY_CLOSED_CONV	I	calculated	—	Count of inquiries with VBFA refs = item count (fully converted)
WF_SO_OPEN_CNT	I	calculated	—	Count of sales orders with 0 VBFA deliveries (open sales orders)
WF_SO_PARTIAL_DEL	I	calculated	—	Count of sales orders with VBFA deliveries < item count (partial deliveries)
WF_SO_CLOSED_DEL	I	calculated	—	Count of sales orders with VBFA deliveries = item count (fully delivered)
WF_DEL_OPEN_CNT	I	calculated	—	Count of deliveries with 0 VBFA billing refs (open deliveries)
WF_DEL_PARTIAL_BILL	I	calculated	—	Count of deliveries with VBFA billing refs < item count (partial billing)
WF_DEL_CLOSED_BILL	I	calculated	—	Count of deliveries with VBFA billing refs = item count (fully billed)

WF_OVERDUE_SO_CNT	I	calculated	—	Count of sales orders where requested date < current date and no VBFA delivery exists
WF_OVERDUE_DEL_CNT	I	calculated	—	Count of deliveries where PGI date < current date and no posting date exists

Structure Name: ZSD_RK_CUST_DASH_TREND_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_DOC_MONTH	CHAR6	calculated	—	Document month in YYYYMM format
WF_INQ_COUNT	I	calculated	—	Number of inquiries for this month (aggregated from VBAK-ERDAT)
WF_SO_COUNT	I	calculated	—	Number of sales orders for this month (aggregated from VBAK-ERDAT)
WF_DEL_COUNT	I	calculated	—	Number of deliveries for this month (aggregated from LIKP-ERDAT)

Structure Name: ZSD_RK_CUST_DASH_TOP_PROD_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_MATNR	MATNR	LIPS	MATNR	Material Number (from LIPS)
WF_MAKTX	MAKTX	MAKT	MAKTX	Material Description (language-dependent, from MAKT)
WF_QTY_SOLD	MENGE_D	LIPS	LFIMG	Total Qty Sold (aggregated LFIMG across all selected deliveries)
WF_NET_VAL	NETWR	VBAP (calculated via VBFA)	NETPR × delivered qty	Net value for that material (sum of LFIMG × VBAP-NETPR via VBFA link)

FUNCTION MODULE CODE:

```
FUNCTION zfm_sd_rk_get_cust_dashboard.
```

```
* "-----
```

```
* " "Local Interface:
```

```
* " IMPORTING
```

```
* " VALUE (IV_KUNNR) TYPE KUNNR
```

```
* " VALUE (IV_DATE_FROM) TYPE DATS DEFAULT 00000000
```



```

*"      VALUE (IV_DATE_TO) TYPE  DATS DEFAULT 99991231

*"      EXPORTING

*"      VALUE (ES_KPI_DATA) TYPE  ZSD_RK_CUST_DASH_KPI_S

*"      VALUE (ET_TRENDS) TYPE  ZSD_RK_CUST_DASH_TREND_T

*"      VALUE (ET_TOP_PROD) TYPE  ZSD_RK_CUST_DASH_TOP_PROD_T

*"      VALUE (EV_MESSAGE) TYPE  STRING

*"-----

IF IV_KUNNR IS INITIAL.

    ev_message = 'Customer Number is Required'.

ENDIF.

DATA: loc_date_from TYPE d,

      loc_date_to   TYPE d.

" Defaults for dates if empty

loc_date_from = COND #( WHEN iv_date_from IS INITIAL THEN '00000000' ELSE
iv_date_from ).

loc_date_to   = COND #( WHEN iv_date_to   IS INITIAL THEN '99991231' ELSE
iv_date_to   ).

*-----

* Data holders for KPI counts & sums

*-----

DATA: loc_inq_count      TYPE i VALUE 0,

      loc_inq_netval     TYPE netwr VALUE 0,

      loc_so_count       TYPE i VALUE 0,

      loc_so_netval      TYPE netwr VALUE 0,

      loc_del_count      TYPE i VALUE 0,

      loc_inq_open_cnt   TYPE i VALUE 0,

      loc_inq_partial_cnt TYPE i VALUE 0,

      loc_inq_closed_cnt TYPE i VALUE 0,

      loc_so_open_cnt    TYPE i VALUE 0,

```

```

loc_so_partial_cnt      TYPE i VALUE 0,
loc_so_closed_cnt       TYPE i VALUE 0,
loc_del_open_cnt        TYPE i VALUE 0,
loc_del_partial_cnt     TYPE i VALUE 0,
loc_del_closed_cnt      TYPE i VALUE 0,

loc_overdue_so_cnt      TYPE i VALUE 0,
loc_overdue_del_cnt     TYPE i VALUE 0,
loc_total_del_days      TYPE i VALUE 0,
loc_actual_del_count    TYPE i VALUE 0.

```

*-----

** Internal tables for documents*

*-----

```

DATA: it_inquiries      TYPE TABLE OF vbak,
      it_sales_orders   TYPE TABLE OF vbak,
      it_deliveries     TYPE TABLE OF likp,
      it_vbfa           TYPE TABLE OF vbfa.

```

*-----

** Select document headers only once per type*

*-----

```

SELECT * FROM vbak INTO TABLE it_inquiries
WHERE kunnr = iv_kunnr
      AND auart = 'AF'
      AND erdat BETWEEN loc_date_from AND loc_date_to.

```

```

SELECT * FROM vbak INTO TABLE it_sales_orders
WHERE kunnr = iv_kunnr
      AND auart = 'TA'
      AND erdat BETWEEN loc_date_from AND loc_date_to.

```

```

SELECT * FROM likp INTO TABLE it_deliveries

```

```
WHERE kunnr = iv_kunnr

AND erdat BETWEEN loc_date_from AND loc_date_to.
```

" Fetch VBFA entries related to these docs

```
SELECT * FROM vbfa INTO TABLE it_vbfa

WHERE vbelv IN (

    SELECT vbeln FROM vbak

    WHERE kunnr = iv_kunnr

    AND auart IN ( 'AF', 'TA' )

    AND erdat BETWEEN loc_date_from AND loc_date_to

)

OR vbelv IN (

    SELECT vbeln FROM likp

    WHERE kunnr = iv_kunnr

    AND erdat BETWEEN loc_date_from AND loc_date_to

).
```

*-----

** KPI: Count inquiries & net values*

*-----

```
loc_inq_count = lines( it_inquiries ).

LOOP AT it_inquiries ASSIGNING FIELD-SYMBOL(<fs_inq>).

    loc_inq_netval = loc_inq_netval + <fs_inq>-netwr.

ENDLOOP.
```

*-----

** KPI: Count sales orders & net values*

*-----

```
loc_so_count = lines( it_sales_orders ).

LOOP AT it_sales_orders ASSIGNING FIELD-SYMBOL(<fs_so>).

    loc_so_netval = loc_so_netval + <fs_so>-netwr.

ENDLOOP.
```

```

*-----
* KPI: Count deliveries
*-----

loc_del_count = lines( it_deliveries ).

*-----
* Create hashed VBFA for fast lookup
*-----

DATA: ht_vbfa TYPE HASHED TABLE OF vbfa WITH UNIQUE KEY vbelv vbtyp_v
vbtyp_n.

LOOP AT it_vbfa INTO DATA(ls_vbfa).

    INSERT ls_vbfa INTO TABLE ht_vbfa.

ENDLOOP.

*-----
* Funnel: Inquiries → SO
*-----

LOOP AT it_inquiries ASSIGNING FIELD-SYMBOL(<fs_inq2>).

    SELECT COUNT(*) INTO @DATA(loc_inq_item_cnt)
        FROM vbap WHERE vbeln = @<fs_inq2>-vbeln.

    SELECT COUNT(*) INTO @DATA(loc_inq_ref_cnt)
        FROM vbfa
        WHERE vbelv = @<fs_inq2>-vbeln
            AND vbtyp_v = 'A' " Inquiry
            AND vbtyp_n = 'C'. " SO

    IF loc_inq_ref_cnt = 0.

        loc_inq_open_cnt = loc_inq_open_cnt + 1.

    ELSEIF loc_inq_ref_cnt < loc_inq_item_cnt.

        loc_inq_partial_cnt = loc_inq_partial_cnt + 1.

```

```
ELSE.

    loc_inq_closed_cnt = loc_inq_closed_cnt + 1.

ENDIF.
```

```
ENDLOOP.
```

```
*-----
```

```
* Funnel: SO → Deliveries
```

```
*-----
```

```
LOOP AT it_sales_orders ASSIGNING FIELD-SYMBOL(<fs_so2>).
```

```
    SELECT COUNT(*) INTO @DATA(loc_so_item_cnt)
        FROM vbap WHERE vbeln = @<fs_so2>-vbeln.
```

```
    SELECT COUNT(*) INTO @DATA(loc_so_deliv_cnt)
        FROM vbfa
        WHERE vbelv = @<fs_so2>-vbeln
            AND vbtyp_v = 'C' " SO
            AND vbtyp_n = 'J'. " Delivery
```

```
    IF loc_so_deliv_cnt = 0.

        loc_so_open_cnt = loc_so_open_cnt + 1.

    ELSEIF loc_so_deliv_cnt < loc_so_item_cnt.

        loc_so_partial_cnt = loc_so_partial_cnt + 1.

    ELSE.

        loc_so_closed_cnt = loc_so_closed_cnt + 1.

    ENDIF.
```

```
ENDLOOP.
```

```
*-----
```

```
* Delivery billing status
```

```
*-----
```

```
LOOP AT it_deliveries ASSIGNING FIELD-SYMBOL(<fs_del2>).
```

```
    SELECT COUNT(*) INTO @DATA(loc_bill_count)
```

```
    FROM vbfa
```

```
    WHERE vbelv = @<fs_del2>-vbeln
```

```
        AND vbtyp_v = 'J' " Delivery
```

```
        AND vbtyp_n = 'M'. " Billing doc
```

```
    SELECT COUNT(*) INTO @DATA(loc_items_count)
```

```
    FROM lips WHERE vbeln = @<fs_del2>-vbeln.
```

```
    IF loc_bill_count = 0.
```

```
        loc_del_open_cnt = loc_del_open_cnt + 1.
```

```
    ELSEIF loc_bill_count < loc_items_count.
```

```
        loc_del_partial_cnt = loc_del_partial_cnt + 1.
```

```
    ELSE.
```

```
        loc_del_closed_cnt = loc_del_closed_cnt + 1.
```

```
    ENDIF.
```

```
ENDLOOP.
```

```
*-----
```

```
* KPI: Overdue Sales Orders
```

```
*-----
```

```
LOOP AT it_sales_orders ASSIGNING FIELD-SYMBOL(<fs_so3>).
```

```
    SELECT SINGLE vdatu
```

```
    FROM vbak
```

```
    WHERE vbeln = @<fs_so3>-vbeln
```

```
    INTO @DATA(loc_req_date).
```

```
    IF loc_req_date IS NOT INITIAL AND loc_req_date < sy-datum.
```

```

        READ TABLE ht_vbfa WITH KEY vbelv = <fs_so3>-vbeln vbttyp_v = 'C'
vbttyp_n = 'J' TRANSPORTING NO FIELDS.

        IF sy-subrc <> 0.

            loc_overdue_so_cnt = loc_overdue_so_cnt + 1.

        ENDIF.

    ENDIF.

ENDLOOP.

*-----
* KPI: Overdue Deliveries and Average Delivery Days
*-----

LOOP AT it_deliveries ASSIGNING FIELD-SYMBOL(<fs_del>).

    IF <fs_del>-wadat IS NOT INITIAL AND

        <fs_del>-wadat < sy-datum AND

        <fs_del>-wadat_ist IS INITIAL.

        loc_overdue_del_cnt = loc_overdue_del_cnt + 1.

    ENDIF.

    " Average delivery days for PGI done deliveries

    IF <fs_del>-wadat_ist IS NOT INITIAL.

        READ TABLE ht_vbfa ASSIGNING FIELD-SYMBOL(<fs_vbfa>)

            WITH KEY vbeln = <fs_del>-vbeln vbttyp_v = 'C' vbttyp_n = 'J'.

        IF sy-subrc = 0.

            SELECT SINGLE vdatu

                FROM vbak

                WHERE vbeln = @<fs_vbfa>-vbelv

                    INTO @DATA(loc_so_req_date).

            IF loc_so_req_date IS NOT INITIAL.

```

```

        DATA(loc_diff_days) = <fs_del>-wadat_ist - loc_so_req_date.

        loc_total_del_days = loc_total_del_days + loc_diff_days.

        loc_actual_del_count = loc_actual_del_count + 1.

    ENDIF.

ENDIF.

ENDIF.

ENDLOOP.

*-----
* Assign KPI results to export structure
*-----

    IF loc_actual_del_count > 0.

        es_kpi_data-WF_AVG_DELIVERY_DAYS = loc_total_del_days /
loc_actual_del_count.

    ELSE.

        es_kpi_data-WF_AVG_DELIVERY_DAYS = 0.

    ENDIF.


es_kpi_data-wf_inquiry_count      = loc_inq_count.
es_kpi_data-wf_inquiry_net_val   = loc_inq_netval.
es_kpi_data-wf_so_count          = loc_so_count.
es_kpi_data-wf_so_net_val        = loc_so_netval.
es_kpi_data-wf_del_count         = loc_del_count.


es_kpi_data-wf_inquiry_open_cnt   = loc_inq_open_cnt.
es_kpi_data-wf_inquiry_partial_conv = loc_inq_partial_cnt.
es_kpi_data-wf_inquiry_closed_conv = loc_inq_closed_cnt.
es_kpi_data-wf_so_open_cnt        = loc_so_open_cnt.
es_kpi_data-wf_so_partial_DEL     = loc_so_partial_cnt.
es_kpi_data-wf_so_closed_DEL      = loc_so_closed_cnt.
es_kpi_data-wf_del_open_cnt       = loc_del_open_cnt.

```



```

es_kpi_data-wf_del_partial_bill  = loc_del_partial_cnt.
es_kpi_data-wf_del_closed_bill   = loc_del_closed_cnt.

es_kpi_data-wf_overdue_so_cnt    = loc_overdue_so_cnt.
es_kpi_data-wf_overdue_del_cnt   = loc_overdue_del_cnt.

*-----
* Trend Chart: Monthly Volumes (Inquiries, SOs, Deliveries)
*-----

TYPES: BEGIN OF ty_trend,
        wf_doc_month TYPE char6,
        wf_inq_count TYPE i,
        wf_so_count  TYPE i,
        wf_del_count TYPE i,
    END OF ty_trend.

DATA: it_trend_temp TYPE SORTED TABLE OF ty_trend WITH UNIQUE KEY
wf_doc_month,

        wa_trend_temp TYPE ty_trend,
        wa_trend_out  TYPE zsd_rk_cust_dash_trend_s.

FIELD-SYMBOLS: <fs_trend> TYPE ty_trend.

" Inquiries by month

LOOP AT it_inquiries ASSIGNING FIELD-SYMBOL(<fs_inq4>).
    DATA(loc_month) = <fs_inq4>-erdat+0(6). " YYYYMM format

    READ TABLE it_trend_temp ASSIGNING <fs_trend> WITH KEY wf_doc_month =
loc_month.

    IF sy-subrc = 0.

        <fs_trend>-wf_inq_count = <fs_trend>-wf_inq_count + 1.

    ELSE.

        CLEAR wa_trend_temp.

        wa_trend_temp-wf_doc_month = loc_month.

```

```

wa_trend_temp-wf_inq_count = 1.

INSERT wa_trend_temp INTO TABLE it_trend_temp.

ENDIF.

ENDLOOP.

```

" Sales Orders by month

```

LOOP AT it_sales_orders ASSIGNING FIELD-SYMBOL(<fs_so4>).

    loc_month = <fs_so4>-erdat+0(6).

    READ TABLE it_trend_temp ASSIGNING <fs_trend> WITH KEY wf_doc_month =
loc_month.

    IF sy-subrc = 0.

        <fs_trend>-wf_so_count = <fs_trend>-wf_so_count + 1.

    ELSE.

        CLEAR wa_trend_temp.

        wa_trend_temp-wf_doc_month = loc_month.

        wa_trend_temp-wf_so_count = 1.

        INSERT wa_trend_temp INTO TABLE it_trend_temp.

    ENDIF.

ENDLOOP.

```

" Deliveries by month

```

LOOP AT it_deliveries ASSIGNING FIELD-SYMBOL(<fs_del4>).

    loc_month = <fs_del4>-erdat+0(6).

    READ TABLE it_trend_temp ASSIGNING <fs_trend> WITH KEY wf_doc_month =
loc_month.

    IF sy-subrc = 0.

        <fs_trend>-wf_del_count = <fs_trend>-wf_del_count + 1.

    ELSE.

        CLEAR wa_trend_temp.

        wa_trend_temp-wf_doc_month = loc_month.

        wa_trend_temp-wf_del_count = 1.

        INSERT wa_trend_temp INTO TABLE it_trend_temp.

    ENDIF.

ENDLOOP.

```

" Transfer to export table ET_TRENDS

LOOP AT it_trend_temp INTO wa_trend_temp.

CLEAR wa_trend_out.

wa_trend_out-wf_doc_month = wa_trend_temp-wf_doc_month.

wa_trend_out-wf_inq_count = wa_trend_temp-wf_inq_count.

wa_trend_out-wf_so_count = wa_trend_temp-wf_so_count.

wa_trend_out-wf_del_count = wa_trend_temp-wf_del_count.

APPEND wa_trend_out TO et_trends.

ENDLOOP.

*-----

** Top Products: Based on Delivered Quantity and Net Value*

*-----

TYPES: BEGIN OF ty_prod_agg,

matnr TYPE matnr,

maktx TYPE maktx,

qty_sold TYPE menge_d,

net_val TYPE netwr,

END OF ty_prod_agg.

DATA: it_prod_agg TYPE STANDARD TABLE OF ty_prod_agg WITH DEFAULT KEY,

wa_prod_agg TYPE ty_prod_agg,

wa_top_prod TYPE zsd_rk_cust_dash_top_prod_s.

FIELD-SYMBOLS: <fs_prod> TYPE ty_prod_agg.

DATA: it_lips TYPE TABLE OF lips,

wa_lips TYPE lips.

" Get delivery items

SELECT * FROM lips INTO TABLE it_lips

FOR ALL ENTRIES IN it_deliveries

```
WHERE vbeln = it_deliveries-vbeln.
```

```
" Aggregate quantity and value by material
```

```
LOOP AT it_lips INTO wa_lips.
```

```
READ TABLE it_prod_agg ASSIGNING <fs_prod>
```

```
WITH KEY matnr = wa_lips-matnr.
```

```
IF sy-subrc = 0.
```

```
<fs_prod>-qty_sold = <fs_prod>-qty_sold + wa_lips-lfimg.
```

```
ELSE.
```

```
CLEAR wa_prod_agg.
```

```
wa_prod_agg-matnr = wa_lips-matnr.
```

```
wa_prod_agg-qty_sold = wa_lips-lfimg.
```

```
APPEND wa_prod_agg TO it_prod_agg ASSIGNING <fs_prod>.
```

```
ENDIF.
```

```
" Get referenced Sales Order item to read VBAP-NETPR
```

```
READ TABLE it_vbfa ASSIGNING FIELD-SYMBOL(<fs_vbfa3>)
```

```
WITH KEY vbeln = wa_lips-vbeln
```

```
posnn = wa_lips-posnr
```

```
vbtyp_v = 'C'
```

```
vbtyp_n = 'J'.
```

```
IF sy-subrc = 0.
```

```
SELECT SINGLE netpr INTO @DATA(loc_netpr)
```

```
FROM vbap
```

```
WHERE vbeln = @<fs_vbfa3>-vbelv
```

```
AND posnr = @<fs_vbfa3>-posnv.
```

```
IF sy-subrc = 0 AND loc_netpr IS NOT INITIAL.
```

```
<fs_prod>-net_val = <fs_prod>-net_val + ( wa_lips-lfimg * loc_netpr  
) .
```

ENDIF.

ENDIF.

ENDLOOP.

" Enrich with material description

LOOP AT it_prod_agg ASSIGNING <fs_prod>.

SELECT SINGLE maktx INTO <fs_prod>-maktx

FROM makt

WHERE matnr = <fs_prod>-matnr

AND spras = sy-langu.

ENDLOOP.

" Sort by net value descending and transfer to export

SORT it_prod_agg BY net_val DESCENDING.

LOOP AT it_prod_agg INTO wa_prod_agg.

CLEAR wa_top_prod.

wa_top_prod-wf_matnr = wa_prod_agg-matnr.

wa_top_prod-wf_maktx = wa_prod_agg-maktx.

wa_top_prod-wf_qty_sold = wa_prod_agg-qty_sold.

wa_top_prod-wf_net_val = wa_prod_agg-net_val.

APPEND wa_top_prod TO et_top_prod.

ENDLOOP.

ev_message = 'Dashboard data Fetched Successfully'.

ENDFUNCTION.

Finance

Credit and Debit Memo

Access Url:

http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_fi_rk_get_cus_t_memo?sap-client=100

Header Structure Name: ZFI_RK_MEMO_HEADER_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_VBELN	VBELN	VBRK	VBELN	Memo Document Number (Billing Document)
WF_FKDAT	FKDAT	VBRK	FKDAT	Billing Date
WF_KUNAG	KUNAG	VBRK	KUNAG	Sold-To Party (Customer who owns the memo)
WF_WAERK	WAERK	VBRK	WAERK	Currency
WF_NETWR	NETWR	VBRK	NETWR	Total Net Amount (from header, before item-level summation)
WF_FKART	FKART	VBRK	FKART	Billing Type (e.g., G2 for Credit Memo, L2 for Debit Memo)
WF_VKORG	VKORG	VBRK	VKORG	Sales Organization
WF_VTWEG	VTWEG	VBRK	VTWEG	Distribution Channel
WF_SPART	SPART	VBRK	SPART	Division
WF_FKSTO	FKSTO	VBRK	FKSTO	Clearing Status ('X' = Reversed, blank = Active)
WF_STATUS	CHAR10	— (calculated) —	'REVERSED' when FKSTO = 'X', else 'ACTIVE'	
WF_VKORG_TXT	VTEXT	TVKOT	VTEXT	Sales Organization Text (language-dependent description)
WF_VTWEG_TXT	VTEXT	TVTWT	VTEXT	Distribution Channel Text (language-dependent description)
WF_SPART_TXT	VTEXT	TSPAT	VTEXT	Division Text (language-dependent description)
WF_FKART_TXT	CHAR10	— (calculated) —	'CREDIT' when FKART = 'G2', else 'DEBIT'	

Item Structure Name: ZFI_RK_MEMO_ITEM_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_VBELN	VBELN	VBRP	VBELN	Memo Document Number (Billing Document)
WF_POSNR	POSNR	VBRP	POSNR	Memo Item Number
WF_MATNR	MATNR	VBRP	MATNR	Material Number
WF_ARKTX	ARKTX	VBRP	ARKTX	Item Description
WF_FKIMG	FKIMG	VBRP	FKIMG	Billing Quantity
WF_VRKME	VRKME	VBRP	VRKME	Sales Unit
WF_ITEM_NETWR	NETWR	VBRP	NETWR	Item Net Amount (net value for this line)
WF_PSTYV	PSTYV	VBRP	PSTYV	Item Category (Billing relevance)
WF_VGBEL	VGBEL	VBRP	VGBEL	Reference Sales Order Number
WF_VGPOS	VGPOS	VBRP	VGPOS	Reference Sales Order Item Number
WF_MWSKZ	MWSKZ	VBRP	MWSKZ	Tax Code
WF_MATWA	MATWA	VBRP	MATWA	Material Group
WF_MEINS	MEINS	VBRP	MEINS	Base Unit of Measure

FUNCTION MODULE CODE:

```

FUNCTION zfm_fi_rk_get_cust_memo.
* "-----
* " "Local Interface:
* "  IMPORTING
* "    VALUE(IV_KUNAG) TYPE KUNAG
* "    VALUE(IV_DATE_FROM) TYPE FKDAT DEFAULT 00000000
* "    VALUE(IV_DATE_TO) TYPE FKDAT DEFAULT 99991231
* "    VALUE(IV_SORT_NEWEST) TYPE STRING OPTIONAL
* "    VALUE(IV_PAGE_NUM) TYPE I DEFAULT 1
* "    VALUE(IV_PAGE_SIZE) TYPE I DEFAULT 10
* "    VALUE(IV_VBELN) TYPE VBELN_VF OPTIONAL
* "    VALUE(IV_BILLING_TYPE) TYPE FKART OPTIONAL
* "    VALUE(IV_STATUS) TYPE STRING OPTIONAL
* "  EXPORTING
* "    VALUE(ET_MEMO_HEADERS) TYPE ZFI_RK_MEMO_HEADER_T
* "    VALUE(ET_MEMO_ITEMS) TYPE ZFI_RK_MEMO_ITEM_T
* "    VALUE(EV_TOTAL_RECORDS) TYPE I
* "    VALUE(EV_MESSAGE) TYPE STRING
* "-----

IF iv_kunag IS INITIAL.
  ev_message = 'Customer number is required'.
  RETURN.
ENDIF.

```

```

TYPES: BEGIN OF ty_vbrk,
        vbeln TYPE vbeln,
        fkdat TYPE fkdat,
        kunag TYPE kunag,
        waerk TYPE waerk,
        netwr TYPE netwr,
        fkart TYPE fkart,
        vkorg TYPE vkorg,
        vtweg TYPE vtweg,
        spart TYPE spart,
        fksto TYPE fksto,
END OF ty_vbrk.

```

```

TYPES: BEGIN OF ty_vbrp,
        vbeln TYPE vbeln,
        posnr TYPE posnr,
        matnr TYPE matnr,
        arktx TYPE arktx,
        fking TYPE fking,
        vrkme TYPE vrkme,
        netwr TYPE netwr,
        pstyv TYPE pstyv,
        vgbel TYPE vgbel,
        vgpos TYPE posnr,
        mwskz TYPE mwskz,
        matwa TYPE matwa,
        meins TYPE meins,
END OF ty_vbrp.

```

```

DATA: it_vbrk      TYPE TABLE OF ty_vbrk,
      it_vbrp      TYPE TABLE OF ty_vbrp,
      wa_header    TYPE zfi_rk_memo_header_s,
      wa_item       TYPE zfi_rk_memo_item_s,
      lt_temp_items TYPE zfi_rk_memo_item_t.

```

```

DATA(loc_date_from) = COND fkdat( WHEN iv_date_from IS INITIAL THEN
'00000000' ELSE iv_date_from ).
DATA(loc_date_to)   = COND fkdat( WHEN iv_date_to IS INITIAL THEN
'99991231' ELSE iv_date_to ).
DATA(loc_offset)    = ( iv_page_num - 1 ) * iv_page_size.

```

```

IF iv_vbeln IS NOT INITIAL.
    SELECT vbeln, fkdat, kunag, waerk, netwr, fkart,
           vkorg, vtweg, spart, fksto
    FROM vbrk
    WHERE vbeln = @iv_vbeln
           AND kunag = @iv_kunag
           AND ( fkart = @iv_billing_type OR @iv_billing_type IS INITIAL AND
fkart IN ('G2', 'L2') )
    INTO TABLE @it_vbrk.

```

```

ELSEIF iv_status IS INITIAL.
    IF iv_sort_newest = 'X'.
        SELECT vbeln, fkdat, kunag, waerk, netwr, fkart,
               vkorg, vtweg, spart, fksto
        FROM vbrk
        WHERE kunag = @iv_kunag
               AND fkdat BETWEEN @loc_date_from AND @loc_date_to

```



```

        AND ( fkart = @iv_billing_type OR @iv_billing_type IS INITIAL AND
fkart IN ('G2', 'L2') )
        ORDER BY fkdat DESCENDING
        INTO TABLE @it_vbrk
        UP TO @iv_page_size ROWS
        OFFSET @loc_offset.
ELSE.
    SELECT vbeln, fkdat, kunag, waerk, netwr, fkart,
           vkorg, vtweg, spart, fksto
    FROM vbrk
    WHERE kunag = @iv_kunag
           AND fkdat BETWEEN @loc_date_from AND @loc_date_to
           AND ( fkart = @iv_billing_type OR @iv_billing_type IS INITIAL AND
fkart IN ('G2', 'L2') )
        ORDER BY fkdat ASCENDING
        INTO TABLE @it_vbrk
        UP TO @iv_page_size ROWS
        OFFSET @loc_offset.
ENDIF.
ELSE.

    IF iv_sort_newest = 'X'.
        SELECT vbeln, fkdat, kunag, waerk, netwr, fkart,
               vkorg, vtweg, spart, fksto
        FROM vbrk
        WHERE kunag = @iv_kunag
               AND fkdat BETWEEN @loc_date_from AND @loc_date_to
               AND ( fkart = @iv_billing_type OR @iv_billing_type IS INITIAL AND
fkart IN ('G2', 'L2') )
            ORDER BY fkdat DESCENDING
            INTO TABLE @it_vbrk.

    ELSE.
        SELECT vbeln, fkdat, kunag, waerk, netwr, fkart,
               vkorg, vtweg, spart, fksto
        FROM vbrk
        WHERE kunag = @iv_kunag
               AND fkdat BETWEEN @loc_date_from AND @loc_date_to
               AND ( fkart = @iv_billing_type OR @iv_billing_type IS INITIAL AND
fkart IN ('G2', 'L2') )
            ORDER BY fkdat ASCENDING
            INTO TABLE @it_vbrk.
    ENDIF.
    SELECT vbeln, fkdat, kunag, waerk, netwr, fkart,
           vkorg, vtweg, spart, fksto
    FROM vbrk
    WHERE kunag = @iv_kunag
           AND fkdat BETWEEN @loc_date_from AND @loc_date_to
           AND ( fkart = @iv_billing_type OR @iv_billing_type IS INITIAL AND
fkart IN ('G2', 'L2') )
        INTO TABLE @it_vbrk.
ENDIF.

    IF it_vbrk IS INITIAL.
        ev_message = 'No memo documents found'.
        ev_total_records = 0.
        RETURN.
    ENDIF.

```

```

IF iv_vbeln IS NOT INITIAL.
    ev_total_records = lines( it_vbrk ).
ELSE.
    SELECT COUNT(*)
    FROM vbrk
    WHERE kunag = @iv_kunag
        AND fkdat BETWEEN @loc_date_from AND @loc_date_to
        AND ( fkart = @iv_billing_type OR @iv_billing_type IS INITIAL AND
fkart IN ('G2', 'L2') )
    INTO @ev_total_records.
ENDIF.

SELECT vbeln, posnr, matnr, arktx, fking, vrkme, netwr,
       pstyv, vgbel, vgpos, mwszk, matwa, meins
FROM vbrp
INTO TABLE @it_vbrp
FOR ALL ENTRIES IN @it_vbrk
WHERE vbeln = @it_vbrk-vbeln.

LOOP AT it_vbrk ASSIGNING FIELD-SYMBOL(<fs_vbrk>).
    DATA: lv_vkorg_txt TYPE tvkot-vtext,
           lv_vtweg_txt TYPE tvttw-vtext,
           lv_spart_txt TYPE tspat-vtext.

    SELECT SINGLE vtext INTO @lv_vkorg_txt FROM tvkot
        WHERE vkorg = @<fs_vbrk>-vkorg AND spras = @sy-langu.

    SELECT SINGLE vtext INTO @lv_vtweg_txt FROM tvttw
        WHERE vtweg = @<fs_vbrk>-vtweg AND spras = @sy-langu.

    SELECT SINGLE vtext INTO @lv_spart_txt FROM tspat
        WHERE spart = @<fs_vbrk>-spart AND spras = @sy-langu.

    wa_header = VALUE #(
        wf_vbeln      = <fs_vbrk>-vbeln
        wf_fkdat      = <fs_vbrk>-fkdat
        wf_kunag      = <fs_vbrk>-kunag
        wf_waerk      = <fs_vbrk>-waerk
        wf_netwr      = <fs_vbrk>-netwr
        wf_fkart      = <fs_vbrk>-fkart
        wf_vkorg      = <fs_vbrk>-vkorg
        wf_vtweg      = <fs_vbrk>-vtweg
        wf_spart      = <fs_vbrk>-spart
        wf_fksto      = <fs_vbrk>-fksto
        wf_status     = COND #( WHEN <fs_vbrk>-fksto = 'X' THEN 'REVERSED' ELSE
'ACTIVE' )
        wf_vkorg_txt  = lv_vkorg_txt
        wf_vtweg_txt  = lv_vtweg_txt
        wf_spart_txt  = lv_spart_txt
        wf_fkart_txt  = COND #( WHEN <fs_vbrk>-fkart = 'G2' THEN 'CREDIT' ELSE
'DEBIT' )
    ).

    IF iv_status IS NOT INITIAL AND wa_header-wf_status <> iv_status.
        CONTINUE.
    ENDIF.

```

```

CLEAR lt_temp_items.
LOOP AT it_vbrp ASSIGNING FIELD-SYMBOL(<fs_vbrp>)
    WHERE vbeln = <fs_vbrk>-vbeln.

    wa_item = VALUE #(
        wf_vbeln   = <fs_vbrp>-vbeln
        wf_posnr   = <fs_vbrp>-posnr
        wf_matnr   = <fs_vbrp>-matnr
        wf_arktx   = <fs_vbrp>-arktx
        wf_fkimg   = <fs_vbrp>-fkimg
        wf_vrkme   = <fs_vbrp>-vrkme
        wf_item_netwr = <fs_vbrp>-netwr
        wf_pstyv   = <fs_vbrp>-pstyv
        wf_vgbel   = <fs_vbrp>-vgbel
        wf_vgpos   = <fs_vbrp>-vgpos
        wf_mwskz   = <fs_vbrp>-mwskz
        wf_matwa   = <fs_vbrp>-matwa
        wf_meins   = <fs_vbrp>-meins
    ).
    APPEND wa_item TO lt_temp_items.
ENDLOOP.

APPEND wa_header TO et_memo_headers.
APPEND LINES OF lt_temp_items TO et_memo_items.
ENDLOOP.

IF iv_status IS NOT INITIAL.
    ev_total_records = lines( et_memo_headers ).
    IF ev_total_records = 0.
        CLEAR et_memo_items.
        ev_message = 'No memos found for specified status'.
    ELSE.
        ev_message = |{ ev_total_records } memo(s) retrieved|.
    ENDIF.
ELSE.
    ev_message = |{ lines( et_memo_headers ) } memo(s) retrieved|.
ENDIF.

ENDFUNCTION.

```

Payments and Aging

Access Url:

http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_fi_rk_customere_pay_aging?sap-client=100

Structure: ZFI_RK_PAYMENTS_AGING_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_VBELN	VBELN	VBRK	VBELN	Billing Document Number
WF_FKART	FKART	VBRK	FKART	Billing Type (e.g., invoice, credit memo)
WF_FKDAT	FKDAT	VBRK	FKDAT	Billing Date
WF_KUNAG	KUNAG	VBRK	KUNAG	Sold-To Party (Customer)
WF_WAERK	WAERK	VBRK	WAERK	Currency
WF_NETWR	NETWR	VBRK	NETWR	Net Amount of the Billing Document
WF_ZTERM	ZTERM	VBRK	ZTERM	Payment Terms Key
WF_ZTERM_TXT	VTEXT	TVZBT	VTEXT	Payment Terms Description (language-dependent, from TVZBT)
WF_XBLNR	XBLNR	VBRK	XBLNR	Reference Document Number (e.g., the external reference)
WF_DUE_DATE	FAEDE-Netdt	calculated via FAEDE	Netdt	Due Date (determined by function DETERMINE_DUE_DATE; if error, defaults to FKDAT)
WF_AGING_DAYS	I	calculated	—	Days Past Due (SY-DATUM minus Due Date; zero if negative)
WF_AGING_BUCKET	STRING	calculated	—	Aging Bucket ("Current", "1-30 Days", "31-60 Days", "61-90 Days", "90+ Days")
WF_AGING_STATUS	STRING	calculated	—	Aging Status ("OPEN" if not yet past due, "OVERDUE" if Due Date < SY-DATUM)
WF_CLEARING_DATE	D	BSEG	AUGDT	Clearing Date (from BSEG-AUGDT; blank if not cleared)
WG_CLEARED	ABAP_BOOL (CHAR1)	calculated	—	Cleared Flag ('X' if a clearing date exists, blank otherwise)
WF_VBELN_SO	VBELN_VA	VBFA	VBELV	Reference Sales Order Number (if billing document is linked via VBFA)

Table Type: ZFI_RK_PAYMENTS_AGING_T Line Type ZFI_RK_PAYMENTS_AGING_S

FUNCTION MODULE FOR PAYMENTS AND AGING:

```
FUNCTION ZFM_FI_RK_GET_CUST_PAY_AGING .
*-----
*""Local Interface:
*  IMPORTING
*    VALUE(IV_KUNAG) TYPE KUNAG
*    VALUE(IV_DATE_FROM) TYPE FKDAT DEFAULT 00000000
*    VALUE(IV_DATE_TO) TYPE FKDAT DEFAULT 99991231
*    VALUE(IV_SORT_NEWEST) TYPE STRING OPTIONAL
*    VALUE(IV_PAGE_NUM) TYPE I DEFAULT 1
*    VALUE(IV_PAGE_SIZE) TYPE I DEFAULT 10
*    VALUE(IV_VBELN) TYPE VBELN_VF OPTIONAL
*    VALUE(IV_AGING_STATUS) TYPE STRING OPTIONAL
*  EXPORTING
*    VALUE(ET_PAYMENTS_AGING) TYPE ZFI_RK_PAYMENTS_AGING_T
*    VALUE(EV_TOTAL_RECORDS) TYPE I
*    VALUE(EV_MESSAGE) TYPE STRING
*-----

DATA: loc_date_from      TYPE fkdat,
      loc_date_to        TYPE fkdat,
      loc_offset          TYPE i,
      loc_due_date        TYPE faede,
      loc_aging_days      TYPE i,
      loc_aging_bucket    TYPE string,
      loc_aging_status    TYPE string,
      loc_cleared          TYPE abap_bool,
      loc_clearing_date   TYPE d,
      loc_zterm_text      TYPE string,
      loc_vbeln_so        TYPE vbeln_va,
      loc_bukrs           TYPE buksr,
      loc_gjahr           TYPE gjahr,
      loc_bseg            TYPE bseg.

DATA: it_vbrk            TYPE STANDARD TABLE OF vbrk,
      it_payment_terms   TYPE STANDARD TABLE OF tvzbt.

FIELD-SYMBOLS: <fs_vbrk> TYPE vbrk,
               <fs_term> TYPE tvzbt.

IF iv_kunag IS INITIAL.
  ev_message = 'Customer number is required'.
  RETURN.
ENDIF.

loc_date_from = COND #( WHEN iv_date_from IS INITIAL THEN '00000000' ELSE
iv_date_from ).
loc_date_to   = COND #( WHEN iv_date_to   IS INITIAL THEN '99991231' ELSE
iv_date_to ).
loc_offset = ( iv_page_num - 1 ) * iv_page_size.

IF iv_vbeln IS NOT INITIAL.
  SELECT vbeln, fkart, fkdat, kunag, waerk, netwr, zterm, xblnr, buksr,
gjahr
FROM vbrk
```

```

        INTO CORRESPONDING FIELDS OF TABLE @it_vbrk
        WHERE kunag = @iv_kunag
            AND vbeln = @iv_vbeln
            AND fkdat BETWEEN @loc_date_from AND @loc_date_to.
    ELSEIF iv_aging_status IS INITIAL.
        IF iv_sort_newest = 'X'.
            SELECT vbeln, fkart, fkdat, kunag, waerk, netwr, zterm, xblnr, bukr,
gjahr
                FROM vbrk
                WHERE kunag = @iv_kunag
                    AND fkdat BETWEEN @loc_date_from AND @loc_date_to
                ORDER BY fkdat DESCENDING
                INTO CORRESPONDING FIELDS OF TABLE @it_vbrk
                UP TO @iv_page_size ROWS
                OFFSET @loc_offset.
        ELSE.
            SELECT vbeln, fkart, fkdat, kunag, waerk, netwr, zterm, xblnr, bukr,
gjahr
                FROM vbrk
                WHERE kunag = @iv_kunag
                    AND fkdat BETWEEN @loc_date_from AND @loc_date_to
                ORDER BY fkdat ASCENDING
                INTO CORRESPONDING FIELDS OF TABLE @it_vbrk
                UP TO @iv_page_size ROWS
                OFFSET @loc_offset.
        ENDIF.
    ELSE.
        IF iv_sort_newest = 'X'.
            SELECT vbeln, fkart, fkdat, kunag, waerk, netwr, zterm, xblnr, bukr,
gjahr
                FROM vbrk
                INTO CORRESPONDING FIELDS OF TABLE @it_vbrk
                WHERE kunag = @iv_kunag
                    AND fkdat BETWEEN @loc_date_from AND @loc_date_to
                ORDER BY fkdat DESCENDING.
        ELSE.
            SELECT vbeln, fkart, fkdat, kunag, waerk, netwr, zterm, xblnr, bukr,
gjahr
                FROM vbrk
                INTO CORRESPONDING FIELDS OF TABLE @it_vbrk
                WHERE kunag = @iv_kunag
                    AND fkdat BETWEEN @loc_date_from AND @loc_date_to
                ORDER BY fkdat ASCENDING.
        ENDIF.
    ENDIF.

    IF it_vbrk IS INITIAL.
        ev_message = 'No billing documents found'.
        ev_total_records = 0.
        RETURN.
    ENDIF.

    IF iv_vbeln IS NOT INITIAL.
        ev_total_records = lines( it_vbrk ).
    ELSEIF iv_aging_status IS INITIAL.
        SELECT COUNT(*) FROM vbrk
            INTO @ev_total_records
            WHERE kunag = @iv_kunag

```

```

        AND fkdat BETWEEN @loc_date_from AND @loc_date_to.
ELSE.
    ev_total_records = lines( et_payments_aging ).
ENDIF.

SELECT *
FROM tvzbt
INTO TABLE @it_payment_terms
WHERE spras = @sy-langu.

LOOP AT it_vbrk ASSIGNING <fs_vbrk>.

    CLEAR: loc_bseg, loc_due_date, loc_clearing_date, loc_cleared.

    loc_bseg-zterm = <fs_vbrk>-zterm.

    DATA(lv_faede) = VALUE faede(
        zfbdt = <fs_vbrk>-fkdat
        rebzg = <fs_vbrk>-vbeln
        koart = 'D'
    ).

    CALL FUNCTION 'DETERMINE_DUE_DATE'
        EXPORTING
            i_faede          = lv_faede
            i_bseg           = loc_bseg
        IMPORTING
            e_faede          = loc_due_date
        EXCEPTIONS
            account_type_not_supported = 1
            OTHERS           = 2.

    IF sy-subrc <> 0.
        loc_due_date-netdt = <fs_vbrk>-fkdat.
    ENDIF.

    loc_aging_days = sy-datum - loc_due_date-netdt.
    IF loc_aging_days < 0.
        loc_aging_days = 0.
    ENDIF.

    loc_aging_bucket = COND string(
        WHEN loc_aging_days = 0          THEN 'Current'
        WHEN loc_aging_days <= 30       THEN '1-30 Days'
        WHEN loc_aging_days <= 60       THEN '31-60 Days'
        WHEN loc_aging_days <= 90       THEN '61-90 Days'
        ELSE '90+ Days'
    ).

    loc_aging_status = COND string(
        WHEN loc_due_date-netdt < sy-datum THEN 'OVERDUE'
        ELSE 'OPEN'
    ).

    IF iv_aging_status IS NOT INITIAL AND iv_aging_status <>
loc_aging_status.
        CONTINUE.
    ENDIF.

```

```

CLEAR loc_zterm_text.
READ TABLE it_payment_terms ASSIGNING <fs_term>
  WITH KEY zterm = <fs_vbrk>-zterm.
IF sy-subrc = 0.
  loc_zterm_text = <fs_term>-vtext.
ENDIF.

CLEAR loc_vbeln_so.
SELECT SINGLE vbelv
  INTO @loc_vbeln_so
  FROM vbfa
  WHERE vbeln = @<fs_vbrk>-vbeln
    AND vbtyp_v = 'C'.

" New: Get clearing status from BSEG
SELECT SINGLE augdt
  INTO @loc_clearing_date
  FROM bseg
  WHERE bukrs = @<fs_vbrk>-bukrs
    AND belnr = @<fs_vbrk>-vbeln.

IF loc_clearing_date IS INITIAL OR loc_clearing_date = '00000000'.
  loc_cleared = abap_false.
ELSE.
  loc_cleared = abap_true.
ENDIF.

APPEND VALUE zfi_rk_payments_aging_s(
  wf_vbeln      = <fs_vbrk>-vbeln
  wf_fkart      = <fs_vbrk>-fkart
  wf_fkdat      = <fs_vbrk>-fkdat
  wf_kunag      = <fs_vbrk>-kunag
  wf_waerk      = <fs_vbrk>-waerk
  wf_netwr      = <fs_vbrk>-netwr
  wf_zterm      = <fs_vbrk>-zterm
  wf_zterm_txt  = loc_zterm_text
  wf_xblnr      = <fs_vbrk>-xblnr
  wf_due_date   = loc_due_date-netdt
  wf_aging_days = loc_aging_days
  wf_aging_bucket = loc_aging_bucket
  wf_aging_status = loc_aging_status
  wf_clearing_date = loc_clearing_date
  wg_cleared    = loc_cleared
  wf_vbeln_so   = loc_vbeln_so
) TO et_payments_aging.

ENDLOOP.

IF lines( et_payments_aging ) > 0.
  ev_message = |{ lines( et_payments_aging ) } payment aging records
retrieved|.
  IF iv_aging_status IS NOT INITIAL.
    ev_total_records = lines( et_payments_aging ).
  ENDIF.
ELSE.
  ev_message = 'No records match the criteria'.
ENDIF.

```


ENDFUNCTION.

Invoice

Access Link:

http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_fi_rk_get_cust_invoice?sap-client=100

Structure Name: ZFI_RK_INVOICE_HEADER_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_VBELN	VBELN	VBRK	VBELN	Invoice Number (Billing Document)
WF_FKDAT	FKDAT	VBRK	FKDAT	Billing Date
WF_RFBSK	RFBSK	VBRK	RFBSK	Billing Status (e.g., Open, Cleared, Canceled)
WF_RFBSK_TXT	VTEXT	DD07T	DDTEXT	Billing Status Text (retrieved via DD07T for domain RFBSK)
WF_BUKRS	BUKRS	VBRK	BUKRS	Company Code
WF_VKORG	VKORG	VBRK	VKORG	Sales Organization
WF_VTWEG	VTWEG	VBRK	VTWEG	Distribution Channel
WF_SPART	SPART	VBRK	SPART	Division
WF_FKART	FKART	VBRK	FKART	Billing Type (e.g., F2 = Invoice)
WF_WAERK	WAERK	VBRK	WAERK	Currency
WF_NETWR	NETWR	(calculated)	—	Total Net Value (sum of VBRP-NETWR for all items)
WF_MWSBP	MWSBP	(calculated)	—	Total Tax Amount (sum of VBRP-MWSBP for all items)
WF_TOTAL_ITEMS	I	(calculated)	—	Total Number of Items on Invoice (count of VBRP lines)
WF_VBTYP	VBTYP	VBRK	VBTYP	Document Category (e.g., 'M' = Billing Document)
WF_FKSTO	FKSTO	VBRK	FKSTO	Clearing Status (e.g., X = Cleared)
WF_VKORG_TXT	VTEXT	TVKOT	VTEXT	Sales Organization Text (language-dependent description)
WF_VTWEG_TXT	VTEXT	TVTWT	VTEXT	Distribution Channel Text (language-dependent description)
WF_SPART_TXT	VTEXT	TSPAT	VTEXT	Division Text (language-dependent description)
WF_FKART_TXT	VTEXT	TVFKT	VTEXT	Billing Type Text (language-dependent description)

Structure Name: **ZSD_RK_INVOICE_ITEM_S**

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_VBELN	VBELN	VBRP	VBELN	Invoice Number (Billing Document)
WF_POSNR	POSNR	VBRP	POSNR	Invoice Item Number
WF_MATNR	MATNR	VBRP	MATNR	Material Number
WF_ARKTX	ARKTX	VBRP	ARKTX	Item Description
WF_FKIMG	FKIMG	VBRP	FKIMG	Billing Quantity
WF_VRKME	VRKME	VBRP	VRKME	Sales Unit
WF_NETWR	NETWR	VBRP	NETWR	Item Net Value
WF_WAERK	WAERK	VBRP	WAERK	Currency
WF_VBELN_SO	AUBEL	VBRP	AUBEL	Reference Sales Order Number
WF_POSNR_SO	AUPOS	VBRP	AUPOS	Reference Sales Order Item Number
WF_MWSBP	MWSBP	VBRP	MWSBP	Tax Amount for This Item
WF_PSTYV	PSTYV	VBRP	PSTYV	Item Category in Billing (e.g., relevant for pricing/taxes)

Table Type Name:

ZSD_RK_INVOICE_HEADER_T Table type for structure **ZSD_RK_INVOICE_HEADER_S**

Table Type Name:

ZSD_RK_INVOICE_ITEM_T Table type for structure **ZSD_RK_INVOICE_ITEM_S**

FUNCTION MODULE FOR INVOICE DETAILS

```
FUNCTION ZFM_FI_RK_GET_CUST_INVOICE .  
  
* "-----  
  
* " "Local Interface:  
  
* " IMPORTING  
  
* " VALUE(IV_KUNAG) TYPE KUNAG  
  
* " VALUE(IV_DATE_FROM) TYPE FKDAT DEFAULT 00000000  
  
* " VALUE(IV_DATE_TO) TYPE FKDAT DEFAULT 99991231
```

```

*"      VALUE (IV_SORT_NEWEST) TYPE  STRING OPTIONAL
*"      VALUE (IV_PAGE_NUM) TYPE  I DEFAULT 1
*"      VALUE (IV_PAGE_SIZE) TYPE  I DEFAULT 10
*"      VALUE (IV_VBELN) TYPE  VBELN_VF OPTIONAL
*"      VALUE (IV_STATUS) TYPE  STRING OPTIONAL
*"      EXPORTING
*"      VALUE (ET_INVOICE_HEADERS) TYPE  ZFI_RK_INVOICE_HEADER_T
*"      VALUE (ET_INVOICE_ITEMS) TYPE  ZFI_RK_INVOICE_ITEM_T
*"      VALUE (EV_TOTAL_RECORDS) TYPE  I
*"      VALUE (EV_MESSAGE) TYPE  STRING
*"-----

```

" Input validation

```

IF iv_kunag IS INITIAL.
    ev_message = 'Customer number is required'.
    RETURN.
ENDIF.

```

" Prefetch status texts

```

DATA: lt_dd07t TYPE TABLE OF dd07t,
      wa_dd07t TYPE dd07t.

```

```

SELECT * INTO TABLE @lt_dd07t
FROM dd07t
WHERE domname = 'RFBSK'
AND ddlanguage = @sy-langu.

```

" Local variables

```

DATA: loc_date_from TYPE fkdat,
      loc_date_to   TYPE fkdat,
      loc_offset    TYPE i.

```

```
loc_date_from = COND #( WHEN iv_date_from IS INITIAL THEN '00000000' ELSE  
iv_date_from ).
```

```
loc_date_to   = COND #( WHEN iv_date_to   IS INITIAL THEN '99991231' ELSE  
iv_date_to ).
```

```
loc_offset    = ( iv_page_num - 1 ) * iv_page_size.
```

```
" Calculate header totals from items
```

```
DATA: loc_total_items TYPE i VALUE 0,  
      loc_total_netwr TYPE netwr VALUE 0,  
      loc_total_tax   TYPE mwsbp VALUE 0.
```

```
" Determine status mode
```

```
DATA(lv_status_mode) = COND #( WHEN iv_status = 'CANCELLED' OR iv_status =  
'X' THEN 'C' ELSE '' ).
```

```
" Invoice header data
```

```
DATA: it_vbrk TYPE STANDARD TABLE OF vbrk WITH EMPTY KEY.
```

```
" Case 1: Specific invoice requested
```

```
IF iv_vbeln IS NOT INITIAL.
```

```
SELECT * FROM vbrk  
  
INTO TABLE @it_vbrk  
  
WHERE vbeln = @iv_vbeln  
  
AND kunrg = @iv_kunag  
  
AND vbtyp = 'M'  
  
AND fkart = 'F2'.
```

```
" Case 2: Status filter provided - fetch ALL records
```

```
ELSEIF iv_status IS NOT INITIAL.
```

```
IF iv_sort_newest = 'X'.
```

```
SELECT * FROM vbrk  
  
INTO TABLE @it_vbrk  
  
WHERE kunrg = @iv_kunag  
  
AND fkdat BETWEEN @loc_date_from AND @loc_date_to
```

```

        AND vbtyp = 'M'

        AND fkart = 'F2'

    ORDER BY fkdat DESCENDING.

ELSE.

    SELECT * FROM vbrk

    INTO TABLE @it_vbrk

    WHERE kunrg = @iv_kunag

        AND fkdat BETWEEN @loc_date_from AND @loc_date_to

        AND vbtyp = 'M'

        AND fkart = 'F2'

    ORDER BY fkdat ASCENDING.

ENDIF.

```

" Case 3: No status filter - apply pagination

```

ELSE.

    IF iv_sort_newest = 'X'.

        SELECT * FROM vbrk

        WHERE kunrg = @iv_kunag

            AND fkdat BETWEEN @loc_date_from AND @loc_date_to

            AND vbtyp = 'M'

            AND fkart = 'F2'

        ORDER BY fkdat DESCENDING

        INTO TABLE @it_vbrk

        UP TO @iv_page_size ROWS

        OFFSET @loc_offset.

    ELSE.

        SELECT * FROM vbrk

        WHERE kunrg = @iv_kunag

            AND fkdat BETWEEN @loc_date_from AND @loc_date_to

            AND vbtyp = 'M'

            AND fkart = 'F2'

        ORDER BY fkdat ASCENDING

        INTO TABLE @it_vbrk
    
```

```

        UP TO @iv_page_size ROWS

        OFFSET @loc_offset.

    ENDIF.
ENDIF.

" Exit if no data found
IF it_vbrk IS INITIAL.

    ev_message = 'No invoices found for specified criteria'.

    ev_total_records = 0.

    RETURN.
ENDIF.

" Get total record count
IF iv_vbeln IS NOT INITIAL.

    ev_total_records = lines( it_vbrk ).
ELSE.

    SELECT COUNT(*) INTO @ev_total_records

        FROM vbrk

        WHERE kunrg = @iv_kunag

            AND fkdatt BETWEEN @loc_date_from AND @loc_date_to

            AND vbtyp = 'M'

            AND fkart = 'F2'.
ENDIF.

" Fetch invoice items in bulk
DATA: it_vbrp TYPE STANDARD TABLE OF vbrp.

SELECT * INTO TABLE @it_vbrp

    FROM vbrp

    FOR ALL ENTRIES IN @it_vbrk

    WHERE vbeln = @it_vbrk-vbeln.

" Prefetch organization texts
DATA: lt_tvkot TYPE TABLE OF tvkot,
```

```
lt_tvtwt TYPE TABLE OF tvttwt,  
lt_tspat TYPE TABLE OF tspat,  
lt_tvfkkt TYPE TABLE OF tvfkkt.
```

```
SELECT * INTO TABLE @lt_tvkkot  
FROM tvkkot  
FOR ALL ENTRIES IN @it_vbrk  
WHERE vkorg = @it_vbrk-vkorg  
AND spras = @sy-langu.
```

```
SELECT * INTO TABLE @lt_tvtwt  
FROM tvttwt  
FOR ALL ENTRIES IN @it_vbrk  
WHERE vtweg = @it_vbrk-vtweg  
AND spras = @sy-langu.
```

```
SELECT * INTO TABLE @lt_tspat  
FROM tspat  
FOR ALL ENTRIES IN @it_vbrk  
WHERE spart = @it_vbrk-spart  
AND spras = @sy-langu.
```

```
SELECT * INTO TABLE @lt_tvfkkt  
FROM tvfkkt  
FOR ALL ENTRIES IN @it_vbrk  
WHERE fkart = @it_vbrk-fkart  
AND spras = @sy-langu.
```

" Process headers with status filtering

```
LOOP AT it_vbrk ASSIGNING FIELD-SYMBOL(<fs_vbrk>).
```

" Apply status filter

```
IF iv_status IS NOT INITIAL.
```



```

" Handle cancelled status

IF lv_status_mode = 'C'.

    IF <fs_vbrk>-fksto <> 'X'.

        CONTINUE.

    ENDIF.

ENDIF.

ENDIF.

ENDIF.

" Get descriptive texts

DATA(lv_vkorg_txt) = VALUE #( lt_tvkot[ vkorg = <fs_vbrk>-vkorg ]-vtext
OPTIONAL ).

DATA(lv_vtweg_txt) = VALUE #( lt_tvttwt[ vtweg = <fs_vbrk>-vtweg ]-vtext
OPTIONAL ).

DATA(lv_spart_txt) = VALUE #( lt_tspat[ spart = <fs_vbrk>-spart ]-vtext
OPTIONAL ).

DATA(lv_fkart_txt) = VALUE #( lt_tvfkkt[ fkart = <fs_vbrk>-fkart ]-vtext
OPTIONAL ).

DATA(lv_rfbsk_txt) = VALUE #( lt_dd07t[ domvalue_1 = <fs_vbrk>-rfbsk
]-ddtext OPTIONAL ).

" Calculate header totals from items

    loc_total_items = 0.

    loc_total_netwr = 0.

    loc_total_tax    = 0.

    LOOP AT it_vbrp ASSIGNING FIELD-SYMBOL(<fs_vbrp>) WHERE vbeln =
<fs_vbrk>-vbeln.

        loc_total_items += 1.

        loc_total_netwr += <fs_vbrp>-netwr.

        loc_total_tax    += <fs_vbrp>-mwsbp.

    ENDLOOP.

" Build header structure

DATA(wa_header) = VALUE zfi_rk_invoice_header_s(

    wf_vbeln      = <fs_vbrk>-vbeln

```

```

wf_fkdat      = <fs_vbrk>-fkdat
wf_rfbsk      = <fs_vbrk>-rfbsk
wf_rfbsk_txt  = lv_rfbsk_txt
wf_bukrs      = <fs_vbrk>-bukrs
wf_vkorg      = <fs_vbrk>-vkorg
wf_vtweg      = <fs_vbrk>-vtweg
wf_spart      = <fs_vbrk>-spart
wf_fkart      = <fs_vbrk>-fkart
wf_waerk      = <fs_vbrk>-waerk
wf_netwr      = loc_total_netwr
wf_mwsbp      = loc_total_tax
wf_total_items = loc_total_items
wf_vbtyp      = <fs_vbrk>-vbtyp
wf_fksto      = <fs_vbrk>-fksto
wf_vkorg_txt  = lv_vkorg_txt
wf_vtweg_txt  = lv_vtweg_txt
wf_spart_txt  = lv_spart_txt
wf_fkart_txt  = lv_fkart_txt
) .

" Process items for this header

LOOP AT it_vbrp ASSIGNING <fs_vbrp> WHERE vbeln = <fs_vbrk>-vbeln.

  APPEND VALUE zfi_rk_invoice_item_s(

    wf_vbeln      = <fs_vbrp>-vbeln
    wf_posnr      = <fs_vbrp>-posnr
    wf_matnr      = <fs_vbrp>-matnr
    wf_arktx      = <fs_vbrp>-arktx
    wf_fkimg      = <fs_vbrp>-fkimg
    wf_vrkme      = <fs_vbrp>-vrkme
    wf_netwr      = <fs_vbrp>-netwr
    wf_waerk      = <fs_vbrp>-waerk
    wf_vbeln_so   = <fs_vbrp>-aubel
    wf_posnr_so   = <fs_vbrp>-aupos

```

```

        wf_mwsbp      = <fs_vbrp>-mwsbp

        wf_pstyv      = <fs_vbrp>-pstyv

    ) TO et_invoice_items.

ENDLOOP.


    APPEND wa_header TO et_invoice_headers.

ENDLOOP.


" Handle status-based record count

IF iv_status IS NOT INITIAL.

    ev_total_records = lines( et_invoice_headers ).

ENDIF.


" Final output message

IF et_invoice_headers IS INITIAL.

    ev_message = 'No invoices found matching criteria'.

    ev_total_records = 0.

ELSE.

    ev_message = ||{ lines( et_invoice_headers ) } invoices retrieved|.

ENDIF.


ENDFUNCTION.

```

Invoice PDF

Access Url:

http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_fi_rk_get_invoice_pdf?sap-client=100

RFC FUNCTION MODULE:

```
FUNCTION zfm_fi_rk_cust_invoice_pdf.
```

```
*"-----
```

```
*" "Local Interface:
```

```
*"  IMPORTING
```

```
*"      VALUE(IV_KUNAG) TYPE  KUNAG
```

```
*"      VALUE(IV_VBELN) TYPE  VBELN_VF OPTIONAL
```

```
*"  EXPORTING
```

```
*"      VALUE(EV_PDF) TYPE  /SDF/CCM_XSTRING
```

```
*"      VALUE(EV_MESSAGE) TYPE  STRING
```

```
*"-----
```

```
DATA: loc_pdf_xstring TYPE /sdf/ccm_xstring.
```

```
FREE MEMORY ID 'RK_PDF_XSTRING'.
```

```
SUBMIT zfi_rk_invoice_pdf
```

```
  WITH p_kunag = iv_kunag
```

```
  WITH p_vbeln = iv_vbeln
```

```
  AND RETURN.
```

```
IMPORT loc_pdf_xstring FROM MEMORY ID 'RK_PDF_XSTRING'.
```

```
ev_pdf = loc_pdf_xstring.
```

```
IF ev_pdf IS INITIAL.
```

```
  ev_message = 'ERROR in generating Adobe Form and XSTRING is Empty'.
```

```
ELSE.
```

```
  ev_message = 'Adobe Form Generated and XSTRING Created Successfully'.
```

```
ENDIF.
```

ENDFUNCTION.

REPORT FORT INVOKING THE INVOICE FORM:

&-----

*& Report ZFI_RK_INVOICE_PDF

&-----

REPORT zfi_rk_invoice_pdf.

PARAMETERS: p_kunag TYPE kunag,
 p_vbeln TYPE vbeln_vf.

DATA: it_header TYPE zfi_rk_invoice_header_t,
 it_item TYPE zfi_rk_invoice_item_t.

DATA: c_form_name TYPE funcname VALUE 'ZF_FI_RK_CUST_INVOICE',
 loc_fmname TYPE funcname,
 loc_out TYPE sfpoutputparams,
 loc_doc TYPE sfpdocparams,
 wa_result TYPE fpformoutput,
 loc_pdf_xstring TYPE /sdf/ccm_xstring,
 ev_pdf TYPE /sdf/ccm_xstring.

loc_out-nodialog = abap_true.

loc_out-getpdf = abap_true.

loc_out-preview = space.

loc_doc-langu = sy-langu.

loc_doc-country = 'IN'.

CALL FUNCTION 'ZFM_FI_RK_GET_CUST_INVOICE'

EXPORTING

```

        iv_kunag          = p_kunag
*   IV_DATE_FROM          = 00000000
*   IV_DATE_TO            = 99991231
*   IV_SORT_NEWEST        =

        iv_page_num       = 1
        iv_page_size      = 9999
        iv_vbeln          = p_vbeln
*   IV_STATUS             =

```

IMPORTING

```

        et_invoice_headers = it_header
        et_invoice_items   = it_item
*   EV_TOTAL_RECORDS      =
*   EV_MESSAGE            =
.

```

```

SELECT SINGLE name1 FROM kna1

INTO @DATA(loc_name)

WHERE kunnr = @p_kunag.

```

" Get the generated FM name for the Adobe Form

```
CALL FUNCTION 'FP_FUNCTION_MODULE_NAME'
```

EXPORTING

```
        i_name          = c_form_name
```

IMPORTING

```
        e_funcname = loc_fmname.
```

```
IF loc_fmname IS INITIAL.
```

```
    MESSAGE 'Form FM name could not be retrieved' TYPE 'E'.
```

```
ENDIF.
```

" Open the form session

```
CALL FUNCTION 'FP_JOB_OPEN'
```

CHANGING

```
        ie_outputparams = loc_out

```

EXCEPTIONS

```
cancel          = 1
usage_error      = 2
system_error     = 3
internal_error   = 4
OTHERS           = 5.
```

```
IF sy-subrc <> 0.
```

```
  MESSAGE 'Error Occurred While Opening the Adobe Form Session' TYPE 'E'.
```

```
ENDIF.
```

```
CALL FUNCTION loc_fmname
```

EXPORTING

```
/lbcdwb/docparams = loc_doc
it_headers         = it_header
it_items           = it_item
kunnr              = p_kunag
name1              = loc_name
```

IMPORTING

```
/lbcdwb/formoutput = wa_result
```

EXCEPTIONS

```
usage_error       = 1
system_error      = 2
internal_error     = 3
OTHERS             = 4.
```

```
IF sy-subrc <> 0.
```

```
  MESSAGE 'Error Occurred While Opening the Adobe Form Module' TYPE 'E'.
```

```
ENDIF.
```

```
CALL FUNCTION 'FP_JOB_CLOSE'
```

```
* IMPORTING
```

```
*   E_RESULT          =
```

EXCEPTIONS

```
usage_error       = 1
```

```

system_error      = 2

internal_error    = 3

OTHERS            = 4.

IF sy-subrc <> 0.

    MESSAGE 'Error Occurred While Closing the Adobe Form Session' TYPE 'E'.

ENDIF.

" Return the generated PDF

loc_pdf_xstring = wa_result-pdf.

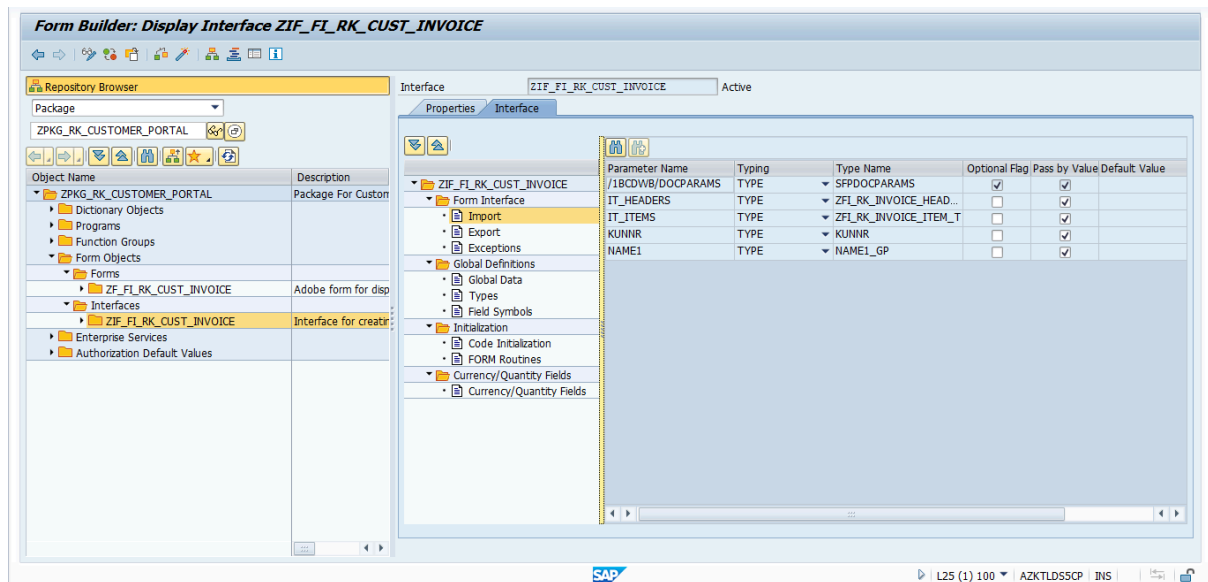
EXPORT loc_pdf_xstring TO MEMORY ID 'RK_PDF_XSTRING'.

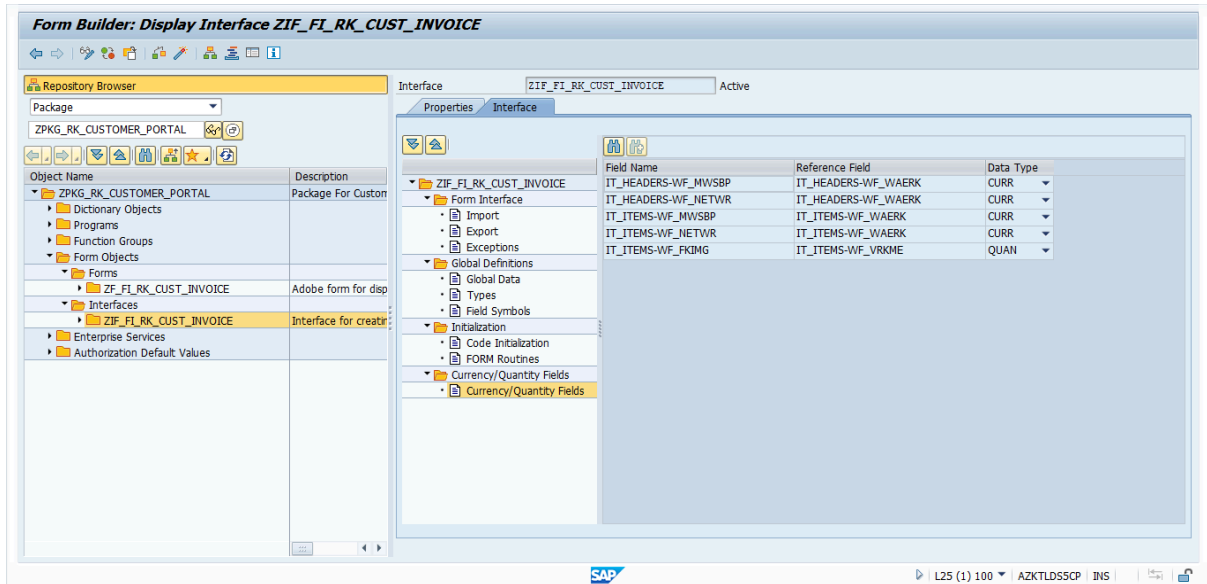
*WRITE: loc_pdf_xstring.

```

ADOBE INTERFACE:

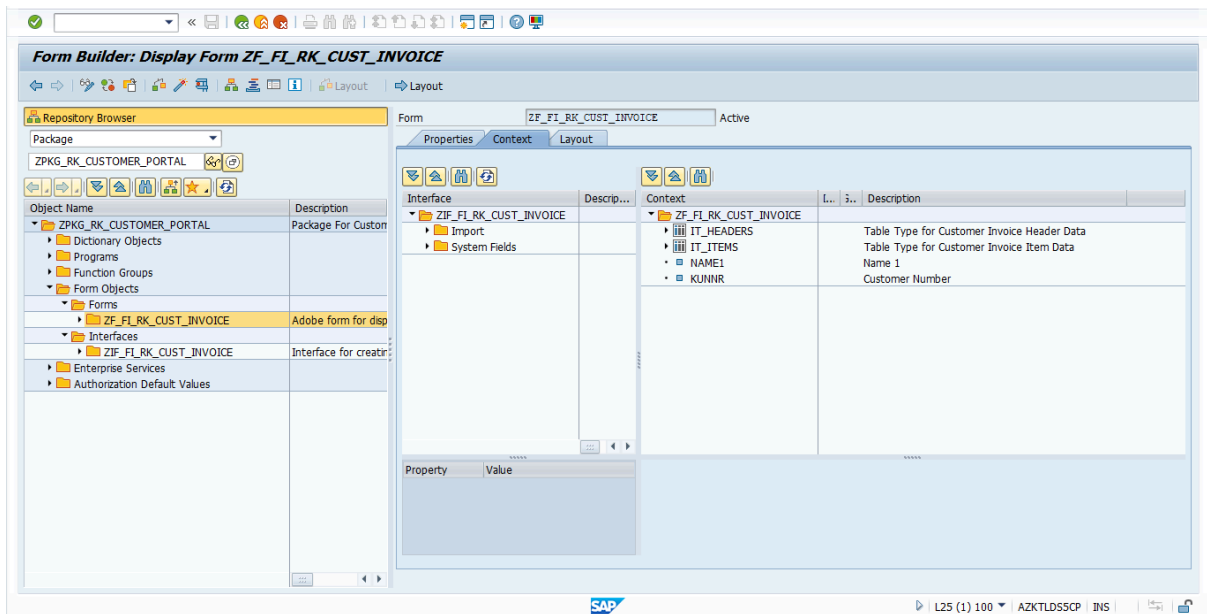
Interface Name: ZIF_FI_RK_CUST_INVOICE





ADOBE FORM:

Form Name: ZF_FI_RK_CUST_INVOICE



OverAll Sales

Access Url:

http://AZKTLDS5CP.kcloud.com:8000/sap/bc/srt/scs/sap/zrfc_fi_get_cust_overall_sale?sap-client=100

Structure: ZFI_RK_CUST_DASH_SUM_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_TOTAL_INV_COUNT	I	calculated	—	Count of invoices (FKART = 'F2').
WF_TOTAL_INV_AMOUNT	NETWR	VBRK	NETWR	Total net amount of all invoices.
WF_TOTAL_CR_MEMO_COUNT	I	calculated	—	Count of credit memos (FKART = 'G2').
WF_TOTAL_CR_MEMO_AMT	NETWR	VBRK	NETWR	Total net amount of credit memos.
WF_TOTAL_DB_MEMO_COUNT	I	calculated	—	Count of debit memos (FKART = 'L2').
WF_TOTAL_DB_MEMO_AMT	NETWR	VBRK	NETWR	Total net amount of debit memos.
WF_TOTAL_PAYMENTS	I	calculated	—	Count of payment documents (invoices only, FKART = 'F2').
WF_PAYMENTS_OPEN	I	calculated	—	Count of open payment documents (due date ≥ today and not cleared).
WF_TOTAL_OPEN_AMOUNT	NETWR	VBRK	NETWR	Sum of net amounts for open or overdue payment documents.
WF_PAYMENTS_OVERDUE	I	calculated	—	Count of overdue payment documents (due date < today and not cleared).
WF_PAYMENTS_CLEARED	I	calculated	—	Count of cleared payment documents.
WF_TOTAL_PAID_AMOUNT	NETWR	VBRK	NETWR	Sum of net amounts for cleared payment documents.
WF_CURRENCY	WAERK	VBRK	WAERK	Currency key used by all summarized documents (from the last processed document).

Structure: ZFI_RK_CUST_DASH_SUM_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_BUCKET_NAME	CHAR20	— (calculated)	—	Aging bucket name (“Current”, “1–30 Days”, “31–60 Days”, “61–90 Days”, “90+ Days”).
WF_AMOUNT	WRBTR	VBRK	NETWR	Total amount for this aging bucket (net amount from billing document).
WF_CURRENCY	WAERK	VBRK	WAERK	Currency key for amounts in this bucket.

Structure: ZFI_RK_CUST_DASH_SUM_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_MATNR	MATNR	VBRP	MATNR	Material number sold.
WF_INVOICE_COUNT	I	— (calculated)	—	Count of invoices in which this material appears.
WF_TOTAL_VALUE	NETWR	VBRP	NETWR	Total net value for this material (sum of VBRP-NETWR on delivered invoices).
WF_CURRENCY	WAERK	VBRP / VBRK	WAERK (VBRK)	Currency key for the total value (from the header VBRK-WAERK).
WF_MAT_DESC	MAKTX	MAKT	MAKTX	Material description (language-dependent) from MAKT.

Structure: ZFI_RK_CUST_DASH_PAY_DETAIL_S

Structure Field	Data Element	Source Table	Field (in Table)	Description
WF_PAYMENT_DOC	VBELN	VBRK	VBELN	Billing document number (payment/invoice).
WF_DUE_DATE	FAEDE-NETDT	calculated	—	Calculated due date (from DETERMINE_DUE_DATE; if error, defaults to VBRK-FKDAT).
WF_STATUS	CHAR10	calculated	—	Payment status (“OPEN” if not yet due, “OVERDUE” if past due and not cleared, “CLEARED” if cleared).
WF_AMOUNT	WRBTR	VBRK	NETWR	Net amount of the billing document.

WF_CURRENCY	WAERK	VBRK	WAERK	Currency key of the billing document.
WF_CLEAR_DATE	AUGDT	BSEG	AUGDT	Clearing date (blank if not cleared).

FUNCTION MODULE CODE:

```
FUNCTION ZFM_FI_RK_GET_CUST_OVERALLSALE .
```

```
*"-----
*""Local Interface:
*
*  IMPORTING
*
*      VALUE (IV_KUNAG) TYPE KUNAG
*
*  EXPORTING
*
*      VALUE (ES_SUMMARY) TYPE ZFI_RK_CUST_DASH_SUM_S
*
*      VALUE (ET_AGING) TYPE ZFI_RK_CUST_DASH_AGING_T
*
*      VALUE (ET_PRODUCT_SUM) TYPE ZFI_RK_CUST_DASH_PROD_SUM_T
*
*      VALUE (ET_PAY_DETAILS) TYPE ZFI_RK_CUST_DASH_PAY_DETAIL_T
*
*      VALUE (EV_MESSAGE) TYPE STRING
*
*-----
```

```
DATA: it_vbrk          TYPE TABLE OF vbrk,
      it_vbrp          TYPE TABLE OF vbrp,
      wa_summary        TYPE zfi_rk_cust_dash_sum_s,
      it_aging          TYPE zfi_rk_cust_dash_aging_t,
      wa_aging          TYPE zfi_rk_cust_dash_aging_s,
      it_product_sum    TYPE zfi_rk_cust_dash_prod_sum_t,
      wa_product_sum    TYPE zfi_rk_cust_dash_prod_sum_s,
      wa_pay_detail      TYPE zfi_rk_cust_dash_pay_detail_s,
      loc_due_date      TYPE faede,
      loc_bseg          TYPE bseg,
      loc_aging_days    TYPE i,
      loc_aging_bucket  TYPE char20,
      loc_aging_status  TYPE char10,
      loc_cleared       TYPE abap_bool,
      loc_amount        TYPE wrbtr,
```

```

loc_currency      TYPE waerk,
lv_faede          TYPE faede,
loc_augdt         TYPE augdt,
lv_gjahr          TYPE gjahr.

```

```

FIELD-SYMBOLS: <fs_vbrk>  TYPE vbrk,
               <fs_vbrp>  TYPE vbrp,
               <fs_aging> TYPE zfi_rk_cust_dash_aging_s,
               <fs_prod>  TYPE zfi_rk_cust_dash_prod_sum_s.

```

```

SELECT * FROM vbrk

      INTO TABLE it_vbrk

      WHERE kunag = iv_kunag

      AND fksto = ''           " Not cancelled

      AND fkart IN ('F2', 'G2', 'L2').

```

```

IF it_vbrk IS INITIAL.

    ev_message = 'No billing data found for customer.'.

    CLEAR: es_summary, et_aging, et_product_sum, et_pay_details.

    RETURN.

ENDIF.

```

```

SELECT * FROM vbrp

      INTO TABLE it_vbrp

      FOR ALL ENTRIES IN it_vbrk

      WHERE vbeln = it_vbrk-vbeln.

```

```

LOOP AT it_vbrk ASSIGNING <fs_vbrk>.

```

```

loc_amount      = <fs_vbrk>-netwr.
loc_currency    = <fs_vbrk>-waerk.
loc_cleared     = abap_false.

```

```

CLEAR: loc_bseg, loc_due_date.

loc_bseg-zterm = <fs_vbrk>-zterm.


lv_faede = VALUE #( zfbdt = <fs_vbrk>-fkdat
                    rebzg = <fs_vbrk>-vbeln
                    koart = 'D' ).


CALL FUNCTION 'DETERMINE_DUE_DATE'

EXPORTING

    i_faede = lv_faede
    i_bseg  = loc_bseg

IMPORTING

    e_faede = loc_due_date

EXCEPTIONS

    OTHERS  = 2.


IF sy-subrc <> 0 OR loc_due_date-netdt IS INITIAL.

    loc_due_date-netdt = <fs_vbrk>-fkdat.

ENDIF.


loc_aging_days = sy-datum - loc_due_date-netdt.

IF loc_aging_days < 0.

    loc_aging_days = 0.

ENDIF.


loc_aging_bucket = COND string(

    WHEN loc_aging_days = 0           THEN 'Current'

    WHEN loc_aging_days <= 30        THEN '1-30 Days'

    WHEN loc_aging_days <= 60        THEN '31-60 Days'

    WHEN loc_aging_days <= 90        THEN '61-90 Days'

    ELSE                             '90+ Days'

) .

```

```
lv_gjahr = <fs_vbrk>-fkdat(4).
```

```
CLEAR loc_augdt.
```

```
SELECT SINGLE augdt
```

```
    INTO loc_augdt
```

```
    FROM bseg
```

```
    WHERE buhrs = <fs_vbrk>-bukrs
```

```
        AND belnr = <fs_vbrk>-vbeln
```

```
        AND gjahr = lv_gjahr
```

```
        AND augdt IS NOT NULL.
```

```
IF sy-subrc = 0.
```

```
    loc_cleared = abap_true.
```

```
ELSE.
```

```
    loc_cleared = abap_false.
```

```
ENDIF.
```

```
IF loc_cleared = abap_true.
```

```
    loc_aging_status = 'CLEARED'.
```

```
ELSEIF loc_due_date-netdt < sy-datum.
```

```
    loc_aging_status = 'OVERDUE'.
```

```
ELSE.
```

```
    loc_aging_status = 'OPEN'.
```

```
ENDIF.
```

```
CASE <fs_vbrk>-fkart.
```

```
    WHEN 'F2'.
```

```
        wa_summary-wf_total_inv_count += 1.
```

```
        wa_summary-wf_total_inv_amount += loc_amount.
```

```
    WHEN 'G2'.
```

```
        wa_summary-wf_total_cr_memo_count += 1.
```

```
        wa_summary-wf_total_cr_memo_amt += loc_amount.
```

```

WHEN 'L2'.

    wa_summary-wf_total_db_memo_count += 1.

    wa_summary-wf_total_db_memo_amt += loc_amount.

ENDCASE.

```

" Payment Status

```

IF <fs_vbrk>-fkart = 'F2'.

    wa_summary-wf_total_payments += 1.

    CASE loc_aging_status.

        WHEN 'OPEN'.

            wa_summary-wf_payments_open += 1.

            wa_summary-wf_total_open_amount += loc_amount.

        WHEN 'OVERDUE'.

            wa_summary-wf_payments_overdue += 1.

            wa_summary-wf_total_open_amount += loc_amount.

        WHEN 'CLEARED'.

            wa_summary-wf_payments_cleared += 1.

            wa_summary-wf_total_paid_amount += loc_amount.

    ENDCASE.

ENDIF.

```

" Aging Status

```

READ TABLE it_aging ASSIGNING <fs_aging> WITH KEY wf_bucket_name =
loc_aging_bucket.

IF sy-subrc = 0.

    <fs_aging>-wf_amount += loc_amount.

ELSE.

    CLEAR wa_aging.

    wa_aging-wf_bucket_name = loc_aging_bucket.

    wa_aging-wf_amount      = loc_amount.

    wa_aging-wf_currency   = loc_currency.

    APPEND wa_aging TO it_aging.

ENDIF.

```



```

IF <fs_vbrk>-fkart = 'F2'.

    LOOP AT it_vbrp ASSIGNING <fs_vbrp> WHERE vbeln = <fs_vbrk>-vbeln.

        READ TABLE it_product_sum ASSIGNING <fs_prod> WITH KEY wf_matnr =
        <fs_vbrp>-matnr.

        IF sy-subrc = 0.

            <fs_prod>-wf_invoice_count += 1.

            <fs_prod>-wf_total_value    += <fs_vbrp>-netwr.

        ELSE.

            CLEAR wa_product_sum.

            wa_product_sum-wf_matnr      = <fs_vbrp>-matnr.

            wa_product_sum-wf_invoice_count = 1.

            wa_product_sum-wf_total_value  = <fs_vbrp>-netwr.

            wa_product_sum-wf_currency    = loc_currency.

            " Get material description

            SELECT SINGLE maktx INTO wa_product_sum-wf_mat_desc

            FROM   maktx

            WHERE  matnr = <fs_vbrp>-matnr

            AND    spras = sy-langu.

            APPEND wa_product_sum TO it_product_sum.

        ENDIF.

    ENDLOOP.

ENDIF.

CLEAR wa_pay_detail.

wa_pay_detail-wf_payment_doc = <fs_vbrk>-vbeln.

wa_pay_detail-wf_due_date    = loc_due_date-netdt.

wa_pay_detail-wf_status      = loc_aging_status.

wa_pay_detail-wf_amount      = loc_amount.

```

```
wa_pay_detail-wf_currency = loc_currency.
```

```
IF loc_cleared = abap_true.
```

```
    wa_pay_detail-wf_clear_date = loc_augdt.
```

```
ELSE.
```

```
    wa_pay_detail-wf_clear_date = ''.
```

```
ENDIF.
```

```
APPEND wa_pay_detail TO et_pay_details.
```

```
ENDLOOP.
```

```
wa_summary-wf_currency = loc_currency.
```

```
es_summary = wa_summary.
```

```
et_aging = it_aging.
```

```
et_product_sum = it_product_sum.
```

```
ev_message = 'OverAll Sales Data Retrived Successfull'.
```

```
ENDFUNCTION.
```